

FORMAL ARCHITECTURE MONOGRAPH

OPERATORIUM

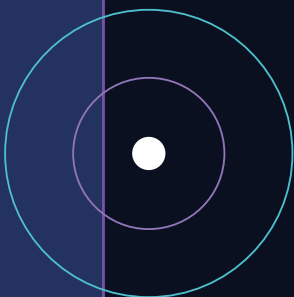
Semantics-Preserving Ludomorphic
Operator Embodiment

A formal framework for operator compilation, immersive field representation and consequence-gated action

Version 1.0.0

Sebastian Klemm

August 2026



OPERATORIUM v1.0.0

Formal Framework for Semantics-Preserving Ludomorphic Operator Embodiment

Autor	Sebastian Klemm
Werkklasse	normative Architekturspezifikation mit Forschungsprogramm
Status	eigenständiger komplementärer Zweig
Schnittstelle	versionierter Außenvertrag zu weltgebundenen Substraten
Normative Sprache	MUSS, DARF NICHT, SOLL, KANN
Referenzversion	1.0.0
Datum	12. August 2026

Werkhinweis

Dieses Dokument beschreibt eine formale Systemarchitektur und ein falsifizierbares Forschungsprogramm. Es behauptet weder empirische Überlegenheit noch bereits realisierte Fähigkeiten. Spielbezogene Anschauungen dienen ausschließlich als Entdeckungsheuristik; die normative Begründung erfolgt durch Typen, Invarianten, Witnesses und Tests.

Kurzfassung

OPERATORIUM spezifiziert einen eigenständigen Herstellungs- und Laufzeitrahmen für menschlich gebundene Feldoperatoren. Aus Personreferenz, Mandat, Kontext, Rechten, Ressourcen und einem modularen Blueprint wird durch einen mehrstufigen Compiler entweder ein versiegeltes Operatorartefakt oder ein lokalisierter Gegenwitness erzeugt. Das Artefakt kann als bewohnbare Operatorgestalt instanziiert werden, deren sichtbare und interaktive Oberfläche keine fiktiven Spielzustände führt, sondern reale Systemzustände semantisch treu, evidenzgebunden und konsequenzsensitiv projiziert.

Der Rahmen grenzt sich von Gamification und Serious Games ab. Er fügt einer Tätigkeit weder Punkte noch künstliche Ziele hinzu und erzeugt keine zweite, folgenlose Welt. Stattdessen definiert er *operative Ludomorphie*: die strukturerhaltende Übertragung erprobter Interaktionsqualitäten – unmittelbares Feedback, räumliche Affordanzen, modular erkennbare Fähigkeiten, Meisterschaft und kohärente Verkörperung – auf ein reales kausales System. Der Mensch bleibt souveräner Ursprung; die Operatorinstanz ist seine begrenzte, widerrufbare und kontextspezifische operative Projektion.

Die Monografie enthält eine getypte Modulalgebra, Blueprint-Normalform, Compilerpipeline, Zustandsautomaten, eine Theorie semantischer Verankerung, eine Konsequenzleiter, Aufmerksamkeit- und Ressourcenmodelle, 39 normative Records, 36 harte Invarianten, Falsifikatoren, Baselines, Blindprotokolle und einen vollständigen Implementierungspfad. Die Kopplung an THRONESCAPE oder ein anderes Wahrnehmungssubstrat erfolgt ausschließlich über einen generischen Portvertrag; beide Zweige bleiben separat implementier-, prüf- und ersetzbar.

Schlüsselwörter. Operatorverkörperung; semantische Treue; Ludomorphie; ökologische Schnittstelle; modulare Komposition; Capability-System; Human-in-the-Loop; Konsequenzgating; Trace; Replay.

Abstract

OPERATORIUM specifies an independent construction and runtime framework for human-bound field operators. A multi-pass compiler consumes a principal reference, mandate, context, permissions, resources, and a modular blueprint, and yields either a sealed operator artifact or a localized counter-witness. The artifact can be instantiated as an inhabitable operator body whose visible and interactive surface carries no fictional game state. Instead, it projects real system state with explicit semantic anchors, evidence, provenance, uncertainty, and consequence.

The framework is distinct from gamification and serious games. It neither decorates work with points nor introduces an inconsequential parallel world. It defines *operational ludomorphy*: a structure-preserving transfer of interaction qualities developed in games – immediate feedback, spatial affordance, legible modular abilities, mastery, and coherent embodiment – onto a real causal system. The human remains the sovereign principal; an operator instance is a bounded, revocable, and context-specific operational projection.

The monograph provides a typed module algebra, blueprint normal form, compiler pipeline, state machines, a theory of semantic anchoring, a consequence ladder, attention and resource models, 39 normative records, 36 hard invariants, falsifiers, baselines, blind protocols, and an implementation path. Coupling to THRONESCAPE or any other world-bound substrate occurs solely through a generic port contract; both branches remain independently implementable, testable, and replaceable.

Inhaltsverzeichnis

Kurzfassung	iii
Abstract	v
Abbildungsverzeichnis	xiii
I Werkgrenze und wissenschaftliche Stellung	1
1 Normativer Beschluss	3
1.1 Werkidentität	3
1.2 Aussageklassen	3
1.3 Normative Sprache	3
1.4 Theorie-Stopp	3
2 Problemklasse und Abgrenzung	5
2.1 Das falsche Problem: Arbeit als Spiel verkleiden	5
2.2 Das eigentliche Problem: operative Distanz	5
2.3 Operative Ludomorphie	6
2.4 Nicht beanspruchte Resultate	6
3 Komplementärer Zweigvertrag	7
3.1 Eigenständigkeit	7
3.2 Generischer Außenvertrag	7
3.3 THRONESCAPE-Adapter	8
3.4 Branch-Merge-Gate	8
II Formale Grundlegung	9
4 Grundsorten, Status und Faktizität	11
4.1 Disjunkte Grundsorten	11
4.2 Statusalgebra	11
4.3 Unbekanntheit als erstklassiger Wert	11
5 Semantische Verankerung und Treue	13
5.1 Projektions- und Handlungssemantik	13
5.2 Informationsverlust und Abstraktion	14
5.3 Immersion als Kopplung, nicht als Grafikdichte	14
6 Person, Bindung und Operatoridentität	15
6.1 Souveräner Ursprung	15
6.2 Bindungstupel	15
6.3 Persistenz und Wechsel	15
6.4 Delegation und Autonomie	16
7 Die Operatormatrix	17
7.1 Modulfamilien	17

7.2	Getypte Modulalgebra	17
7.3	Blueprint als endlicher Hypergraph	17
7.4	Fähigkeitsprofil	18
III	Kompilation und Instanziierung	19
8	Blueprint-Normalform	21
8.1	Kanonische Bestandteile	21
8.2	Normalisierung	21
8.3	Blueprint-Schichten	21
9	Der Operator-Compiler	23
9.1	Compilervertrag	23
9.2	Neun Pflichtpässe	23
9.3	Gegenwitnessprinzip	24
9.4	Stochastische Analysen	24
10	Artefakt, Instanz und Versiegelung	25
10.1	Operatorartefakt	25
10.2	Instanziierung	25
10.3	Lebenszyklusautomat	25
10.4	Hot Swap	25
IV	Verkörperung und ludomorphe Erfahrung	27
11	Der Operatorleib	29
11.1	Verkörperung als geschlossene Regelkopplung	29
11.2	Drei Schichten	29
11.3	Körperkontinuität	30
11.4	Eigenbewegung und Umweltbewegung	30
12	Ludomorphe Projektionsgrammatik	31
12.1	Übertragbare Invarianten	31
12.2	Makroskalige Subtilität	31
12.3	Verbotene Gamification-Substitute	31
12.4	Hyperrealität	32
13	Affordanzen und Interaktionsalgebra	33
13.1	Affordanzrecord	33
13.2	Interaktionsoperatoren	33
13.3	Gestenstabilität	33
13.4	Direkte und makroskopische Führung	34
14	Feld, Karte und Navigation	35
14.1	Operative Feldlandschaft	35
14.2	Vorwärtsbewegung und Altitude	35
14.3	Routen und Passagen	35
14.4	Karte und Territorium	35
14.5	Explorationspolitik	36

15 Ressourcen-, Fähigkeits- und Aufmerksamkeitsökologie	37
15.1 Reale Ressourcenvektoren	37
15.2 Leases statt Besitz	37
15.3 Regeneration und Cooldown	37
15.4 Aufmerksamkeit als harte Ressource	37
15.5 Meisterschaft ohne Grind	37
16 Konsequenzleiter und Wirkungsgrenze	39
16.1 Fünf Regime	39
16.2 Wirkungstoken	39
16.3 Preview und Commit	39
16.4 Reconciliation	40
V Maschineninfrastruktur	41
17 Kanonischer Laufzustand	43
17.1 Zustandstupel	43
17.2 Ein-Wahrheitszustand-Regel	43
17.3 Ereignisatom	43
17.4 Epochen	43
18 Normative Records	45
18.1 Konstitution und Kompilation	45
18.2 Semantik und Verkörperung	45
18.3 Laufzeit und Wirkung	45
18.4 Governance, Trace und Test	46
19 Operatorregistry und Laufnormalform	47
19.1 Registryfamilien	47
19.2 Referenzlauf	47
19.3 Deterministischer Kern	47
20 Ledger, Trace und Replay	49
20.1 Hashverkettetes Ledger	49
20.2 Drei Replaystufen	49
20.3 Gegenfaktisches Replay	49
20.4 Datensparsamkeit	49
21 Sicherheit, Integrität und menschliche Souveränität	51
21.1 Gefahrenmodell	51
21.2 Fail-Closed	51
21.3 Anti-Manipulationsvertrag	51
21.4 Übersteuerung und Exit	51
21.5 Erklärbarkeit	52
VI Invarianten, Falsifikation und Realisierung	53
22 Adamantine Invarianten	55
22.1 Konstitutionsinvarianten	55
22.2 Semantikinvarianten	55

22.3	Compiler- und Kompositionsinvarianten	55
22.4	Wirkungs- und Laufzeitinvarianten	56
22.5	Human- und Governanceinvarianten	56
23	Falsifizierbare Kernthesen	57
23.1	Hypothesenmatrix	57
23.2	Baselinefamilien	57
23.3	Metriksatz	57
23.4	Falsifikatorboxen	58
24	Versiegelte Referenzwelten und Blindprotokoll	59
24.1	Referenzweltfamilien	59
24.2	Blindgenerierung	59
24.3	Ablationen	59
24.4	Abnahmegrenzen	60
25	Referenzimplementierung	61
25.1	Minimaler Systemzuschnitt	61
25.2	Empfohlene technische Grenze	61
25.3	Schema Lock	61
25.4	Eigenschaftstests	61
25.5	Minimale Workbench	62
26	Vertikale Realisierung	63
26.1	Phasenordnung	63
26.2	Sedimentationsgates	63
26.3	Branch-Disziplin	63
26.4	Erster materialisierter Witness	64
27	Closure und Änderungsdisziplin	65
27.1	Closure-Kriterium	65
27.2	Zulässige Änderungsklassen	65
27.3	Was nicht als Fortschritt zählt	65
27.4	Offen, aber nicht theoriebildend	65
VII	Anhänge	67
A	Kompakte Notation	69
B	Port-Normalformen	71
B.1	WorldSnapshot	71
B.2	ActionProposal	71
B.3	EffectReceipt	71
C	Abnahmematrix	73
D	Heuristische Herkunft und formale Reinigung	75
D.1	Motivierende Anschauung	75
E	Wissenschaftliche Einordnung	77
E.1	Beziehung zu etablierten Linien	77

E.2 Abgrenzung zu Digital Twins und Avataren	77
E.3 Abgrenzung zu Serious Games	77
Literaturverzeichnis	81
Sachregister	83

Abbildungsverzeichnis

3.1	Komplementäre Zweigarchitektur. Beide Systeme sind vollständig, aber nur über einen schmalen Außenvertrag gekoppelt.	7
5.1	Semantische Kommutativität: sichtbare Veränderung und reale Zustandsänderung müssen innerhalb deklarierter Toleranz übereinstimmen.	13
9.1	Der Compiler erzeugt entweder ein versiegeltes Operatorartefakt oder einen lokalisierten Gegenwitness.	23
11.1	Verkörperung als gebundene Schichtung: souveräner Ursprung, Operatorleib und semantisch verankerte Projektionshülle.	29
16.1	Konsequenzleiter. Flüssige Exploration wird erst an der faktischen Wirkungsgrenze absichtlich gebremst.	39

Teil I

Werkgrenze und wissenschaftliche Stellung

1 Normativer Beschluss

1.1 Werkidentität

Kanonischer Satz

OPERATORIUM ist ein formaler Rahmen für die Kompilation, Instanziierung, Verkörperung und konsequenzgebundene Führung menschlich autorisierter Feldoperatoren. Er macht reale Systemzustände intuitiv bewohnbar, ohne sie durch fiktive Spielzustände zu ersetzen.

Der Eigenname bezeichnet einen Herstellungs- und Arbeitsraum, kein Akronym. Die normative Einheit ist dieses Dokument. Implementierungen DÜRFEN Produktnamen tragen, MÜSSEN aber ihre Konformität zu einer konkreten OPERATORIUM-Version deklarieren.

1.2 Aussageklassen

Klasse	Geltung	Erkennungsmerkmal
Definition	modellintern kraft Festsetzung	formal nummeriert
Axiom	normative Grundannahme	formal nummeriert
Proposition/Satz	logische Folgerung unter Voraussetzungen	Beweis oder Beweisskizze
Architekturvertrag	Implementierungspflicht	MUSS/DARF NICHT/SOLL/KANN
Empirische Hypothese	durch Versuch entscheidbar	Metrik, Baseline, Falsifikator
Heuristik	nicht normative Entdeckungshilfe	ausdrücklich als solche markiert
Metapher	Anschauung ohne Beweiskraft	kein Typ, kein Witness

1.3 Normative Sprache

MUSS und **DARF NICHT** bezeichnen Konformitätsbedingungen. **SOLL** bezeichnet eine starke Vorgabe, von der nur mit dokumentiertem Gegenargument abgewichen werden darf. **KANN** bezeichnet eine optionale Realisierung. Jede normative Forderung ist einem Record, Operator, Invariant oder Test zugeordnet.

1.4 Theorie-Stopp

Version 1.0.0 schließt die architektonische Grundlegung, sobald (i) alle Grundsarten typisiert, (ii) alle Promotionsgrenzen mit Witnesses versehen, (iii) alle Außenports definiert, (iv) alle Kernthesen falsifizierbar und (v) die erste Referenzimplementierung vollständig spezifiziert sind. Unentschiedene Algorithmen, Parameter und Gestaltungsvarianten bleiben danach Forschungs- oder Implementierungsfragen; sie rechtfertigen keine erneute Metaphorikphase.

2 Problemklasse und Abgrenzung

2.1 Das falsche Problem: Arbeit als Spiel verkleiden

Gamification wird gewöhnlich als Einsatz von Game-Design-Elementen in Nicht-Spiel-Kontexten gefasst (Deterding et al., 2011a,b). Diese Definition ist für OPERATORIUM gerade die Negativgrenze. Ein reales System wird nicht dadurch verständlicher, dass es Punkte, Ranglisten, Streaks, künstliche Knappheit oder kosmetische Progression erhält. Solche Elemente können Motivation verändern, ohne die operative Lesbarkeit zu erhöhen.

Definition 2.1 (Fiktiver Spielzustand). Ein sichtbarer Zustand g ist fiktiv, wenn kein deklarierter Referent x im realen System, kein Ableitungsoperator ρ , keine Unsicherheit und kein Provenienz witness existieren, so dass $g = \rho(x)$ innerhalb einer angegebenen Toleranz gilt.

Axiom 2.2 (Keine fiktive Autorität). Kein fiktiver Spielzustand DARF eine reale Berechtigung, Fähigkeit, Ressource, Gefahr oder Wirkung vortäuschen, verdecken oder ersetzen.

2.2 Das eigentliche Problem: operative Distanz

Zwischen Mensch und komplexem System liegen typischerweise Datenmodelle, Tabellen, Menüs, Konfigurationssprachen, Prozessgrenzen und verzögerte Rückmeldungen. Diese Distanz erzeugt nicht nur Bedienaufwand, sondern trennt Wahrnehmung und Handlung. Ökologische Interfacegestaltung versucht, arbeitsrelevante Constraints direkt wahrnehmbar zu machen (Vicente und Rasmussen, 1992; Vicente, 2002); Direct Manipulation verlangt kontinuierliche Repräsentation, physische statt syntaktische Aktionen und rasches Feedback (Shneiderman, 1983). OPERATORIUM erweitert diese Linie um kompilierte Operatoridentität, explizite Faktizität und eine harte Wirkungsgrenze.

Definition 2.3 (Operative Distanz). Für eine Aufgabe q sei

$$D_{\text{op}}(q) = w_1 N_{\text{trans}} + w_2 L_{\text{fb}} + w_3 H_{\text{hidden}} + w_4 E_{\text{syntax}} + w_5 R_{\text{ambiguity}},$$

wobei N_{trans} die Zahl notwendiger Repräsentationswechsel, L_{fb} die Feedbacklatenz, H_{hidden} die Menge handlungsrelevanter verborgener Zustände, E_{syntax} die syntaktische Belastung und $R_{\text{ambiguity}}$ das Risiko semantischer Fehlinterpretation bezeichnen.

OPERATORIUM minimiert D_{op} nicht blind. An der externen Wirkungsgrenze wird kontrolliert Reibung eingeführt, sofern sie Fehlwirkungen verhindert. Leichtigkeit ist kein Selbstzweck; sie ist nur dann zulässig, wenn Beobachtbarkeit, Kontrolle und Faktizität erhalten bleiben.

2.3 Operative Ludomorphie

Definition 2.4 (Operative Ludomorphie). Operative Ludomorphie ist die semantikerhaltende Abbildung eines realen kausalen Systems auf eine kohärente Wahrnehmungs- und Handlungswelt unter Nutzung spielerisch erprobter Interaktionsformen, ohne Einführung eines unabhängigen Spielziels, einer fiktiven Ressource oder einer folgenlosen Aktionsklasse.

Gamification = Tätigkeit + Spielzeichen, Ludomorphie = Realsystem $\cong_{\varepsilon}$ Interaktionswelt.

Das Symbol $\cong_{\varepsilon}$ bezeichnet keine mathematische Isomorphie aller internen Strukturen, sondern eine später präzierte beobachtungs- und handlungsbezogene Äquivalenz bis zu deklarierter Toleranz.

2.4 Nicht beanspruchte Resultate

OPERATORIUM behauptet nicht, dass räumliche oder spielinspirierte Oberflächen generell besser sind, Immersion stets die Leistung steigert, Menschen fehlerfrei handeln, ein Operatorartefakt eine Person vollständig repräsentiert oder die vorgeschlagenen Metaphern domänenübergreifend invariant sind. Diese Fragen werden als Hypothesen mit Baselines behandelt.

3 Komplementärer Zweigvertrag

3.1 Eigenständigkeit

OPERATORIUM besitzt eigene Grundsorten, eigene Zustandsautomaten, eigene Invarianten, einen eigenen Compiler und einen eigenen Falsifikationsapparat. Es setzt weder Gyrosphären, Rotorfasern noch eine bestimmte Weltmodellarchitektur voraus. Ein weltgebundenes Substrat ist ein austauschbarer Außenpartner.

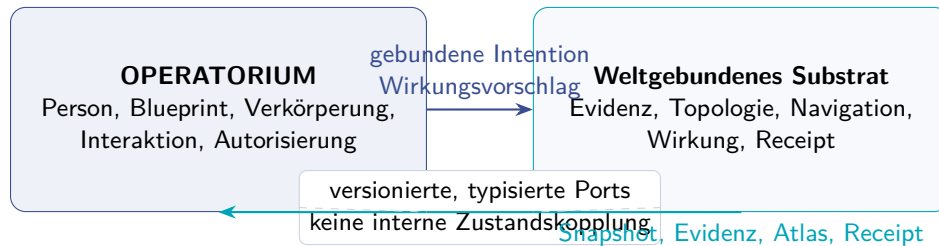


Abbildung 3.1: Komplementäre Zweigarchitektur. Beide Systeme sind vollständig, aber nur über einen schmalen Außenvertrag gekoppelt.

3.2 Generischer Außenvertrag

Der minimale Außenvertrag besteht aus acht Portfamilien:

Port	Richtung	Verpflichtender Inhalt
WorldSnapshot	ein	Zustandsreferenzen, Zeit, Scope, Faktizität, Provenienz
EvidencePacket	ein	Evidenz, Messvertrag, Unsicherheit, Horizon
TopologyAtlas	ein	Regionen, Übergänge, Erreichbarkeit, lokale Höhenfelder
CapabilityOffer	ein	verfügbare Sensor-, Navigations- und Wirkoperatoren
BoundIntent	aus	Personbindung, Mandat, Zielscope, Ablaufzeit
ActionProposal	aus	Effektklasse, Ziel, Vorbedingungen, Budget, Risiko
EffectReceipt	ein	externer Beleg über Versuch und beobachtetes Ergebnis
RevocationNotice	beidseitig	unverzüglicher Entzug von Rechten, Bindung oder Verbindung

Jeder Port MUSS versioniert, kanonisierbar und replayfähig sein. Interne Objektadressen, mutable Pointer oder unbezeugte Seiteneffekte DÜRFEN die Grenze nicht überschreiten.

Proposition 3.1 (Ersetzbarkeit). Erfüllen zwei Außenpartner denselben Portvertrag und sind alle OPERATORIUM-Entscheidungen ausschließlich Funktionen portierter Daten und lokaler Records, dann kann der Partner ausgetauscht werden, ohne die innere Semantik eines bereits versiegelten Blueprints zu verändern.

Beweis. Nach Voraussetzung existiert keine weitere Abhängigkeit. Gleiche kanonische Porteingaben erzeugen daher bis zu explizit deklarierten stochastischen Rändern gleiche Validierungs- und Promotionsentscheidungen. Änderungen der Partnerrealisierung sind beobachtungsäquivalent, sofern sie den Portvertrag erfüllen. $\square$

3.3 THRONESCAPE-Adapter

Ein THRONESCAPE-Adapter KANN dessen Welt-, Evidenz-, Atlas-, Navigations-, Gate- und Receipt-Objekte auf die generischen Ports abbilden. Die Begriffe *Throne*, *Ophan*, *Gyrosphäre* und *Orbitkern* bleiben erlaubte Produkt- oder Integrationsrollen, sind aber keine Grundsorten von OPERATORIUM. Dadurch bleibt das Framework auch ohne THRONESCAPE vollständig.

3.4 Branch-Merge-Gate

Eine spätere Assimilation in einen gemeinsamen Implementierungsmonolithen ist erst zulässig, wenn:

1. beide Referenzkerne separat konform laufen,
2. ihre Porttraces vollständig replayfähig sind,
3. kein gemeinsamer interner Typ zur Konformität erforderlich ist,
4. die gekoppelte Ausführung gegen beide isolierten Baselines getestet wurde,
5. ein Rückbau auf die Portkopplung möglich bleibt.

Teil II

Formale Grundlegung

4 Grundsarten, Status und Faktizität

4.1 Disjunkte Grundsarten

Seien

Principal, Mandate, Context, Right, Resource, Module, Blueprint,
Artifact, Instance, View, Action, Effect, Witness, Trace

disjunkte Grundsarten. Ein Operator ist ein getypter, im Allgemeinen partiell definierter Morphemus

$$f : A \rightarrow B$$

mit deklarierter Domäne, Kodomäne, Vorbedingung, Nachbedingung, Ressourcenwirkung, Effektklasse und Witnessvertrag.

4.2 Statusalgebra

Definition 4.1 (Faktizitätsstatus).

$$\mathbb{F} = \{\text{Specified, Possible, Simulated, Proposed, Authorized, Attempted, Observed, Reconciled, Revoked}\}.$$

Die Elemente sind keine lineare Wahrheitsskala. Zulässige Promotionen bilden einen gerichteten Graphen, dessen Kanten jeweils einen getypten Witness verlangen.

Insbesondere gelten

$$\text{Simulated} \neq \text{Observed}, \quad \text{Authorized} \neq \text{Attempted}, \quad \text{Attempted} \neq \text{Reconciled}.$$

Axiom 4.2 (Explizite Promotion). Kein Objekt darf seinen Faktizitätsstatus implizit durch Rendering, Zeitablauf, Nutzerannahme oder bloße Wiederholung wechseln.

4.3 Unbekanntheit als erstklassiger Wert

Jeder Record, der eine Tatsachenbehauptung trägt, MUSS neben Wert und Typ eine Unsicherheits- oder Unbekanntheitsrepräsentation besitzen. Ein Renderer DARF Unbekanntheit nicht durch einen Defaultwert ersetzen. Fehlende Daten, widersprüchliche Daten und außerhalb des Horizons liegende Daten sind verschiedene Zustände.

Definition 4.3 (Semantisches Tupel). Ein sichtbarer Bedeutungsgegenstand ist

$$s = (r, f, u, p, h, v),$$

mit Referent r , Faktizität $f \in \mathbb{F}$, Unsicherheit u , Provenienz p , Horizon h und Versionsbindung v .

5.1 Projektions- und Handlungssemantik

übersetzt sichtbare Handlungen in Wirkungsvorschläge. Sie führt niemals unmittelbar einen externen Effekt aus.

5 Semantische Verankerung und Treue

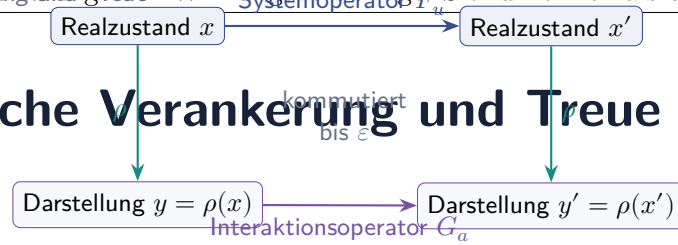


Abbildung 5.1: Semantische Kommutativität: sichtbare Veränderung und reale Zustandsänderung müssen innerhalb deklarierter Toleranz übereinstimmen.

Definition 5.1 (ε -semantische Kommutativität). Für Metriken oder anwendungsspezifische Diskrepanzmaße d_Y gilt eine Aktion a mit interpretiertem Operator $u = \kappa(y, a)$ auf $D \subseteq X$ als ε -treu, wenn

$$\sup_{x \in D} d_Y(\rho(F_u(x)), G_a(\rho(x))) \leq \varepsilon.$$

Axiom 5.2 (Semantischer Anker). Jedes handlungsrelevante sichtbare Objekt MUSS auf ein semantisches Tupel und einen überprüfbaren Projektionsoperator zurückführbar sein.

Definition 5.3 (Sichtvertrag). Ein Sichtvertrag ist

$$V = (D, \rho, d_Y, \varepsilon, \Lambda, \Omega),$$

wobei D der Geltungsbereich, ρ die Projektion, d_Y das Diskrepanzmaß, ε die Toleranz, Λ die deklarierter Informationsverlustmenge und Ω der Faktizitäts- und Provenienzvertrag sind.

Satz 5.4 (Keine stille Bedeutungsverschiebung). Sind alle handlungsrelevanten Objekte durch versionsgebundene Sichtverträge verankert und ist jede Vertragsänderung eine explizite Epochen-grenze, dann kann eine Bedeutungsänderung nicht innerhalb derselben View-Epoche konform verborgen bleiben.

Beweis. Eine Bedeutungsänderung verändert mindestens $D, \rho, d_Y, \varepsilon, \Lambda$ oder Ω . Da diese Größen versionsgebunden und Bestandteil des View-Witnesses sind, erzeugt jede Änderung entweder einen neuen Vertrag oder eine Vertragsverletzung. Beides ist innerhalb derselben Epoche beobachtbar. $\square$

5.2 Informationsverlust und Abstraktion

Eine Projektion darf verlustbehaftet sein. Verboten ist nicht die Abstraktion, sondern die unmarkierte Abstraktion. Der Verlustoperator

$$\lambda_\rho(x) = x / \sim_\rho$$

fasst Zustände zusammen, die dieselbe sichtbare Darstellung erzeugen. Für jede Handlungsklasse muss geprüft werden, ob diese Quotientierung entscheidungsrelevante Unterschiede verschluckt.

Beweisobligation 5.5 (Handlungssuffizienz). Für jede sichtbare Aktion a und alle $x_1 \sim_\rho x_2$ ist nachzuweisen, dass entweder derselbe reale Operator zulässig ist oder die Darstellung vor Aktionsfreigabe verfeinert wird.

5.3 Immersion als Kopplung, nicht als Grafikdichte

Die operative Immersionsqualität einer View wird heuristisch durch

$$I_{\text{op}} = \frac{C_{\text{sem}} F_{\text{near}} K_{\text{sensor}} A_{\text{agency}}}{1 + D_{\text{repr}} + R_{\text{ambiguity}} + L_{\text{latency}}}$$

charakterisiert. Die Terme stehen für semantische Kopplung, Feedbacknähe, sensorimotorische Kohärenz, erlebte Handlungsfähigkeit, Repräsentationsdistanz, Mehrdeutigkeit und Latenz. Die Formel ist ein Messgerüst, kein psychologisches Naturgesetz.

6 Person, Bindung und Operatoridentität

6.1 Souveräner Ursprung

Definition 6.1 (Principal). Ein Principal $p \in P$ ist die rechtlich und operativ maßgebliche Herkunft einer Intention. OPERATORIUM speichert keine ontologische Kopie einer Person, sondern eine minimale, widerrufbare Referenz mit Authentisierungs- und Mandatswitness.

Axiom 6.2 (Nicht-Identität). Für jede Operatorinstanz o und jeden Principal p gilt $o \neq p$. Eine Instanz ist eine begrenzte operative Projektion und darf weder Identität noch Zustimmung des Principals über ihren Mandatsscope hinaus behaupten.

6.2 Bindungstupel

Eine gültige Bindung ist

$$\beta = (p, m, c, \pi, r, \tau, w),$$

mit Principal p , Mandat m , Kontext c , Rechtemenge π , Ressourcenbudget r , Ablaufbedingung τ und Witness w . Die Bindung ist widerrufbar und nicht transitiv, sofern Transitivität nicht ausdrücklich Bestandteil des Mandats ist.

Definition 6.3 (Operative Gestalt). Die operative Gestalt eines Principals im Kontext c_t ist

$$o_t = \text{Instantiate}(p, b, c_t, \pi, r_t),$$

sofern der Compiler ein versiegeltes Artefakt erzeugt und die Laufzeitbindung gültig ist.

6.3 Persistenz und Wechsel

Der Principal kann mehrere Blueprints besitzen; eine Instanz kann ihre Darstellung adaptieren, ohne ihre Artefaktidentität zu verlieren. Unterschieden werden:

- ▶ *Principalpersistenz*: dieselbe autoritative Herkunft,
- ▶ *Artefaktpersistenz*: derselbe versiegelte Blueprint und Compilerwitness,
- ▶ *Instanzpersistenz*: dieselbe Laufzeitidentität,
- ▶ *Viewkontinuität*: fortgesetzte Darstellung innerhalb eines Sichtvertrags.

Proposition 6.4 (Kontextuelle Vielgestaltigkeit). Mehrere Instanzen $o^{(1)}, \dots, o^{(n)}$ desselben Principals verletzen die Principalpersistenz nicht, sofern jede Instanz eine disjunkte oder explizit koordinierte Mandats- und Rechthülle besitzt.

Beweis. Die Identität des Principals ist eine Referenzrelation, nicht die Gleichheit der Instanzzustände. Konfliktfreiheit folgt aus disjunkten Effektbereichen oder einem gemeinsamen Koordinationswitness. $\square$

6.4 Delegation und Autonomie

Automatisierung ist eine endliche Delegation

$$\delta : p \mapsto (\mathcal{A}, \mathcal{C}, \mathcal{B}, \mathcal{T}, \mathcal{R}),$$

mit erlaubten Aktionstypen, Kontexten, Budgets, Zeitgrenzen und Revokationsbedingungen. Eine Instanz besitzt keine offene Generalvollmacht. Jede autonome Schleife MUSS eine endliche Halt-, Degradations- oder Rückfragebedingung besitzen.

7 Die Operatormatrix

7.1 Modulfamilien

Die Operatormatrix ist kein Figurenkatalog, sondern der Raum zulässiger Operatorzusammensetzungen:

$$\mathcal{M} = \mathcal{S} \otimes \mathcal{T} \otimes \mathcal{N} \otimes \mathcal{Z} \otimes \mathcal{E} \otimes \mathcal{V} \otimes \mathcal{P},$$

mit Sensorik $\mathcal{S}$, Transformation $\mathcal{T}$, Navigation $\mathcal{N}$, Stabilisierung $\mathcal{Z}$, Effektoren $\mathcal{E}$, Views $\mathcal{V}$ und Policies $\mathcal{P}$.

7.2 Getypte Modulalgebra

Ein Modul ist

$$m = (id, A, B, f, q, e, h, w),$$

wobei $f : A \rightarrow B$ die Funktion, q der Ressourcenvektor, e die Effektklasse, h der Hazardvertrag und w der Witnessvertrag ist. Module bilden eine partielle symmetrisch-monoidale Kompositionsstruktur: serielle Komposition $g \circ f$ ist nur bei Typ-, Policy- und Witnesskompatibilität definiert; parallele Komposition $f \otimes g$ addiert Budgets und vereinigt Effektflächen.

Definition 7.1 (Kompatibilitätsrelation).

$$K(m_i, m_j, c, \pi) \in \{\top, \perp, ?\}$$

liefert kompatibel, inkompatibel oder unentschieden. Ein unentschiedenes Paar darf nicht als kompatibel kompiliert werden, sofern kein Sandboxvertrag die Unbestimmtheit vollständig isoliert.

Axiom 7.2 (Geschlossene Komposition). Jede ausführbare Kante eines Blueprints MUSS typ-, budget-, rechte-, hazard- und witnesskompatibel sein. Lokale Kompatibilität genügt nicht, wenn globale Zyklen oder gemeinsame Ressourcen neue Konflikte erzeugen.

7.3 Blueprint als endlicher Hypergraph

Ein Blueprint ist

$$B = (V, E, H, I, O, C, \Pi, Q),$$

mit Modulinstanzen V , gerichteten Datenkanten E , höherordentlichen Constraints H , Eingangsports I , Ausgangsports O , Kontextprädikaten C , Policy Π und Budget Q . Er MUSS endlich, versioniert und kanonisierbar sein.

Definition 7.3 (Zulässiger Blueprint). Ein Blueprint ist im Kontext c zulässig, wenn

$$\text{Adm}(B, c, \pi, q) = T \wedge W \wedge P \wedge R \wedge H \wedge S = \top,$$

wobei T Typabschluss, W Witnessabschluss, P Rechteabschluss, R Ressourcenabschluss, H Hazardabschluss und S semantische Sichtabdeckung bezeichnen.

7.4 Fähigkeitsprofil

Aus einem zulässigen Blueprint wird kein skalarer Power-Level abgeleitet. Das Fähigkeitsprofil ist eine partiell geordnete Menge

$$\mathcal{C}_B = \{(a, D_a, \pi_a, q_a, \varepsilon_a, w_a)\},$$

die für jede Fähigkeit ihren Geltungsbereich, Rechte, Ressourcen, Toleranz und Witness nennt. Zwei Blueprints sind im Allgemeinen nicht total vergleichbar.

Proposition 7.4 (Kein ehrlicher Gesamtlevel). Existieren zwei Fähigkeiten a, b mit gegensätzlicher Dominanz zweier Blueprints, dann gibt es ohne externe Nutzenfunktion keinen semantisch neutralen skalaren Rang der Blueprints.

Beweis. Sei B_1 bezüglich a überlegen und B_2 bezüglich b . Jede skalare Ordnung benötigt Gewichte für a, b ; diese Gewichte sind eine externe Nutzenfunktion. Ohne sie ist die Ordnung nicht kanonisch. $\square$

Teil III

Kompilation und Instanziierung

8 Blueprint-Normalform

8.1 Kanonische Bestandteile

Jeder Blueprint MUSS mindestens enthalten:

1. Identität, Version und Hashverfahren,
2. erlaubte Principal- und Kontextklassen,
3. Modulgraph und Porttypen,
4. Mandats- und Rechteanforderungen,
5. Ressourcen- und Aufmerksamkeitsbudgets,
6. Hazard- und Degradationsmodell,
7. Sichtverträge und Interaktionsgrammatik,
8. Effektklassen und Konsequenzregime,
9. Trace-, Replay- und Revokationsvertrag,
10. Abnahmetests und bekannte Residuen.

8.2 Normalisierung

Zwei syntaktisch verschiedene Blueprints können dieselbe operative Struktur ausdrücken. Der Normalisierer N ordnet IDs, expandiert Defaults, kanonisiert Einheiten, sortiert kommutative Mengen und fixiert alle impliziten Versionen. Gefordert ist

$$N(N(B)) = N(B).$$

Definition 8.1 (Artefaktäquivalenz).

$$B_1 \equiv_A B_2 \iff \text{hash}(N(B_1)) = \text{hash}(N(B_2)).$$

Diese Äquivalenz ist stärker als bloße Verhaltensähnlichkeit und schwächer als textuelle Gleichheit.

8.3 Blueprint-Schichten

Die Normalform trennt:

Schicht	Inhalt
Constitution	Principal-Klasse, Mandat, Lebenszyklus, Revokation
Organs	Sensor-, Transformations-, Navigations-, Stabilisierungs- und Effektermodule
Envelope	Rechte, Ressourcen, Hazard, Kontext, Horizon
Embodiment	Körpermodell, Orientierung, View- und Affordanzverträge
Consequence	Promotionspfade, Gates, Receipt und Reconciliation
Evidence	Tests, Witnesses, Residuen, Provenienz

9 Der Operator-Compiler

9.1 Compilervertrag

Der Compiler ist eine deterministisch orchestrierte Folge möglicherweise stochastischer Analysen. Für identische kanonische Eingaben, Seeds, Resolverstände und Policyversionen MUSS er ein byte-identisches Artefakt oder einen äquivalenten Gegenwitness erzeugen.

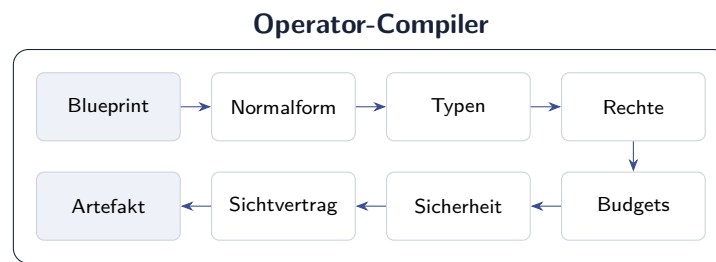


Abbildung 9.1: Der Compiler erzeugt entweder ein versiegeltes Operatorartefakt oder einen lokalisierten Gegenwitness.

9.2 Neun Pflichtpässe

1. **Canonicalize:** Normalform und stabile IDs.
2. **Resolve:** exakte Modul- und Policyversionen.
3. **Type:** lokale und globale Porttypen.
4. **Close Rights:** transitive Effekt- und Datenrechte.
5. **Prove Budgets:** Worst-Case- und Laufzeitgrenzen.
6. **Analyze Hazards:** Hazardpfade, Korrelationen, Common Causes.
7. **Cover Semantics:** Sicht- und Handlungssuffizienz.
8. **Synthesize Runtime:** Automaten, Guards, Degradation, Revokation.
9. **Seal:** Manifest, Hash, Signaturbindung, Testvektoren.

Listing 9.1: Normativer Compilerablauf

```

1 compile(blueprint, context, mandate, rights, resources):
2   b0 := canonicalize(blueprint)
3   b1 := resolve_exact_versions(b0)
4   require type_close(b1)
5   require right_close(b1, mandate, rights)
6   require budget_close(b1, resources)
7   require hazard_close(b1, context)
8   require semantic_cover(b1, context)
9   runtime := synthesize_state_machines(b1)
10  tests    := execute_conformance_vectors(b1, runtime)
11  require tests.all_mandatory_pass
12  return seal(b1, runtime, tests)
13 on failure:
14  return CounterWitness(pass, locus, inputs, violated_contract)

```

9.3 Gegenwitnessprinzip

Ein Compilerfehler ist kein Freitext. Er MUSS mindestens Pass, minimale Fehlerstelle, betroffene Typen oder Constraints, verwendete Versionen, Reproduktionsinput und verletzten Vertrag enthalten. Kann der Fehler nicht lokalisiert werden, ist der Compiler selbst nicht konform.

Satz 9.1 (Fehlschlagtotalität). Wenn jeder Compilerpass auf endlichen Eingaben terminiert und entweder einen transformierten Zustand oder einen Gegenwitness liefert, dann ist compile als Ergebnisfunktion total, auch wenn die Artefakterzeugung partiell ist.

Beweis. Endliche sequentielle Komposition terminierender totaler Ergebnisfunktionen ist total. Ein früher Fehlschlag beendet mit Gegenwitness; andernfalls endet der letzte Pass mit Artefakt oder Gegenwitness. □

9.4 Stochastische Analysen

Optimierer, Solver oder Modelle dürfen stochastisch sein, müssen aber Seed, Modellhash, Samplingvertrag, Konfidenz und deterministischen Akzeptanzguard protokollieren. Ein probabilistischer Vorschlag darf keinen normativen Guard ersetzen.

10 Artefakt, Instanz und Versiegelung

10.1 Operatorartefakt

Ein versiegeltes Artefakt ist

$$A = (B^*, R, P, Q, H, V, S, T, \eta),$$

mit normalisiertem Blueprint B^* , Resolverlock R , Polyclosure P , Budgetbeweis Q , Hazardbericht H , Sichtverträgen V , Automaten S , Testmanifest T und Siegel η .

Axiom 10.1 (Artefaktimmobilität). Ein versiegeltes Artefakt ist unveränderlich. Jede inhaltliche Änderung erzeugt eine neue Artefaktidentität; Laufzeitparameter dürfen nur innerhalb vorab versiegelter Bereiche variieren.

10.2 Instanziierung

Instanziierung bindet ein Artefakt an aktuelle Portzustände, Principalwitness und Ressourcenleases:

$$\text{instantiate} : A \times \beta \times W_t \rightarrow O_t.$$

Sie MUSS alle zeitabhängigen Vorbedingungen erneut prüfen. Ein historisch gültiges Artefakt garantiert keine aktuelle Aktivierbarkeit.

10.3 Lebenszyklusautomat

$$\text{Draft} \rightarrow \text{Validated} \rightarrow \text{Sealed} \rightarrow \text{Bound} \rightarrow \text{Embodied} \rightarrow \text{Active}.$$

Von Active sind nur folgende Übergänge zulässig:

$$\text{Active} \rightarrow \{\text{Degraded}, \text{Suspended}, \text{Quarantined}, \text{Retired}\}.$$

Eine Rückkehr aus Quarantined erfordert einen neuen Außenwitness; Retired ist terminal.

10.4 Hot Swap

Modulwechsel zur Laufzeit ist nur über einen transaktionalen Swap erlaubt:

1. Effektoren einfrieren,
2. Zustand snapshotten,
3. neue Modulversion in Isolation binden,
4. Zustandsmigration validieren,
5. Sichtvertrag und Capability Envelope vergleichen,
6. atomar umschalten oder vollständig zurückrollen.

Beweisobligation 10.2 (Kontinuität). Ein Hot Swap muss nachweisen, dass keine Aktion, Berechtigung, Unsicherheit oder laufende Verpflichtung durch die Migration still verloren geht.

Teil IV

Verkörperung und ludomorphe Erfahrung

11 Der Operatorleib

11.1 Verkörperung als geschlossene Regelkopplung

Eine Operatorinstanz wird nicht dadurch verkörpert, dass sie einen Avatar erhält. Verkörperung liegt vor, wenn Wahrnehmung, Orientierung, Intention, Aktion und Rückmeldung in einer fortsetzbaren, persongebundenen Schleife zusammenfallen:

$$\mathcal{E}_t = (P_t, O_t, V_t, A_t, W_t, \beta_t),$$

mit Principalzustand P_t , Operatorzustand O_t , View V_t , Affordanzen A_t , Weltkopplung W_t und Bindung β_t .

Definition 11.1 (Operative Verkörperung). Eine Instanz ist im Zeitraum $[t_0, t_1]$ operativ verkörpert, wenn

1. ihre Principalbindung durchgehend gültig ist,
2. jede aktive Affordanz auf eine aktuelle Capability zurückführbar ist,
3. Wahrnehmungs- und Wirkungsfeedback innerhalb deklarierter Latenzen eintreffen,
4. Viewkontinuität oder ein sichtbarer Epochenschnitt vorliegt,
5. Verlust der Kopplung in Degradation statt in unbezeugte Fortsetzung führt.

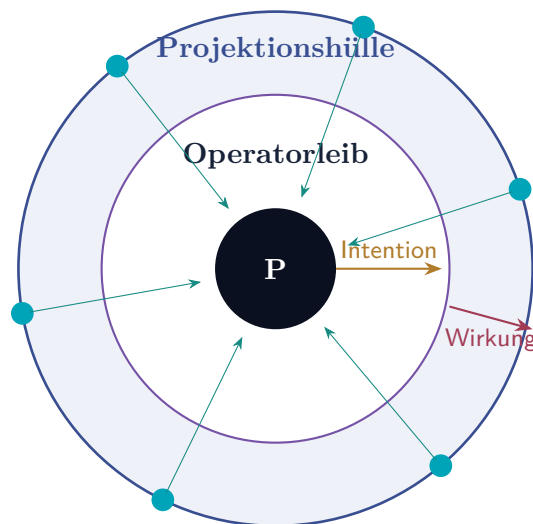


Abbildung 11.1: Verkörperung als gebundene Schichtung: souveräner Ursprung, Operatorleib und semantisch verankerte Projektionshülle.

11.2 Drei Schichten

1. Der **souveräne Kern** trägt Principalbindung, Mandat, Aufmerksamkeit und finalen Autorisierungsstatus.
2. Der **Operatorleib** trägt Module, Fähigkeiten, Ressourcen, Stabilisierung und laufende Zustände.
3. Die **Projektionshülle** trägt semantisch verankerte Ansichten, Affordanzen und Umweltkopplungen.

Eine Produktintegration darf diese Schichten als Throne, Ophan und Gyrosphäre darstellen. Formal entscheidend bleiben Bindung, Operatorgraph und Sichtvertrag.

11.3 Körperkontinuität

Der Operatorleib kann seine sichtbare Geometrie verändern. Kontinuität erfordert nicht Formgleichheit, sondern eine nachvollziehbare Transformationskette

$$V_{t+1} = T_t(V_t), \quad \text{Trace}(T_t) \neq \emptyset.$$

Plötzliche Rekonfigurationen müssen als Modus-, Kontext- oder Epochengrenzen markiert werden. Eine stille Umbelegung derselben Geste oder Form ist unzulässig.

11.4 Eigenbewegung und Umweltbewegung

Views MÜSSEN zwischen mindestens vier Ursachen sichtbarer Bewegung unterscheiden:

- ▶ Eigenbewegung des Principals oder seiner Eingabereferenz,
- ▶ interne Reorientierung des Operatorleibs,
- ▶ Zustandsänderung des abgebildeten Außenfelds,
- ▶ reine Kamerabewegung ohne operative Bedeutung.

Werden diese Bewegungen visuell identisch behandelt, muss die Ursache anderweitig unmittelbar erkennbar sein.

12 Ludomorphe Projektionsgrammatik

12.1 Übertragbare Invarianten

OPERATORIUM übernimmt keine Spielgenres. Es extrahiert Interaktionsinvarianten, die in Strategie-, Aufbau-, Fahrzeug- und Champion-Systemen wiederholt leistungsfähig sind:

Invariante	Operative Funktion	Verbotene Verzerrung
Kohärenter Leib	Fähigkeiten werden als zusammenhängende Gestalt erfahren	Avatar ohne Capability-Bezug
Modularität	Umbau verändert nachweisbar Verhalten	kosmetische Teile mit Scheineffekt
Räumliche Affordanz	Handlungsmöglichkeiten sind an der Lage erkennbar	dekorative Klickfläche ohne Referent
Unmittelbares Feedback	Aktion und Rückmeldung bilden eine Schleife	bestätigende Animation vor Receipt
Meisterschaft	bessere Modelle und Koordination steigern Leistung	künstlicher Grind oder Zeitwall
Makrosteuerung	Ziele werden auf passender Abstraktionsebene gebunden	unsichtbare Vollautonomie
Ressourcenökologie	Engpässe und Flüsse werden wahrnehmbar	fiktive Währung ohne Systemreferent
Fähigkeitsprofil	Reichweite und Grenzen sind im Leib sichtbar	eindimensionaler Power-Level
Exploration	Unbekanntheit wird begehbar, aber nicht erfunden	Karte als behauptete Vollständigkeit

12.2 Makroskalige Subtilität

Ludomorphie ist umso subtiler, je weniger sie als Belohnungsapparat sichtbar wird. Sie darf auf der Makroebene Form, Rhythmus und Kausalgefühl bestimmen, ohne die Aufmerksamkeit auf ein Spielregelwerk zu ziehen. Ein konformes System zeigt daher bevorzugt:

- ▶ echte Abhängigkeiten als räumliche Nähe oder Bindung,
- ▶ echte Engpässe als Verdichtung, Spannung oder Trägheit,
- ▶ echte Unsicherheit als Unschärfe, Mehrfachkontur oder instabile Lage,
- ▶ echte Reichweite als geometrische oder topologische Zugänglichkeit,
- ▶ echte Regeneration als beobachtbaren Wiederherstellungsprozess,
- ▶ echte Kompetenz als vergrößertes validiertes Capability Envelope.

12.3 Verbotene Gamification-Substitute

Folgende Elemente sind standardmäßig nicht konform, sofern kein realer Referent und Sichtvertrag existiert:

- ▶ Punkte, Erfahrungspunkte, Level und Ranglisten,
- ▶ Streaks oder Verlustdrohungen zur Verhaltensbindung,

- ▶ künstliche Cooldowns ohne technische oder regulatorische Ursache,
- ▶ Loot, zufällige Freischaltungen und synthetische Knappheit,
- ▶ Erfolgsanimationen vor externer Bestätigung,
- ▶ anthropomorphe Aussagen über Wissen, Absicht oder Zustimmung der Maschine,
- ▶ verdeckte Schwierigkeitsanpassung, die reale Zustände umdeutet.

Axiom 12.1 (Referententreue). Jede als Ressource, Fähigkeit, Gefahr, Fortschritt oder Erfolg wahrnehmbare Form MUSS einen deklarierten operativen Referenten besitzen oder ausdrücklich als rein ästhetisch und handlungsirrelevant markiert sein.

12.4 Hyperrealität

Hyperrealität bezeichnet hier weder Fotorealismus noch eine stärkere Behauptung als die Evidenz trägt. Sie bezeichnet eine Darstellung, in der operative Beziehungen unmittelbar lesbar sind als in ihrer nativen technischen Form. Eine topologische Risikofläche kann insofern „realer“ bedienbar sein als eine Tabelle, obwohl sie abstrakter aussieht.

Definition 12.2 (Operative Hyperrealität). Eine View ist operativ hyperreal bezüglich einer Aufgabenklasse Q , wenn sie die entscheidungsrelevanten Invarianten des Realsystems erhält und die operative Distanz für Q gegenüber der nativen Baseline reduziert, ohne Faktizitäts- oder Fehlerrate zu verschlechtern.

13 Affordanzen und Interaktionsalgebra

13.1 Affordanzrecord

Eine Affordanz ist

$$a = (id, \sigma, \gamma, \kappa, \pi, q, e, g, w),$$

mit sichtbarer Signatur σ , Geltungsbereich γ , Interpretationsoperator κ , Rechten π , Ressourcen q , Effektklasse e , Guard g und Witnessvertrag w .

Axiom 13.1 (Keine Geisteraffordanz). Eine sichtbare Affordanz darf nicht aktiv erscheinen, wenn ihre Capability, Rechte, Ressourcen oder Vorbedingungen fehlen. Umgekehrt darf eine aktive Wirkungsmöglichkeit nicht vollständig unsichtbar bleiben.

13.2 Interaktionsoperatoren

Die minimale Algebra enthält:

Operator	Semantik
inspect	Evidenz, Provenienz, Horizon und Verlustmenge öffnen
focus	Aufmerksamkeit und Auflösung lokal erhöhen
bind	Intention an Ziel, Scope und Ablaufbedingung koppeln
compose	Module oder Teilpläne typisiert verbinden
probe	informationsgewinnende, wirkungsarme Aktion vorschlagen
simulate	kontrafaktische Entwicklung ohne Außenwirkung erzeugen
compare	Szenarien, Baselines oder Artefakte differenzieren
authorize	vorgeschlagenen Effekt an Principalwitness binden
commit	autorisierten Vorschlag an den Außenpartner übergeben
revoke	Bindung, Lease, Fähigkeit oder laufende Absicht entziehen
reconcile	Receipt gegen Vorschlag, Versuch und Beobachtung prüfen
replay	sichtbare und kausale Folge deterministisch rekonstruieren

13.3 Gestenstabilität

Eine Geste oder Interaktionsform darf innerhalb einer Epoche nicht ihre Effektklasse wechseln. Insbesondere darf dieselbe Geste nicht einmal Navigation und einmal externe Wirkung bedeuten, sofern kein expliziter Moduswitness sichtbar ist.

Proposition 13.2 (Modusfreie Sicherheit). Wenn jede Effektklasse eine disjunkte Affordanzsignatur besitzt und κ diese Signatur typisiert prüft, kann ein reiner Navigationsakt nicht als externer Effekt interpretiert werden.

Beweis. Disjunkte Signaturen implizieren, dass keine Eingabe gleichzeitig den Typen der Navigations- und Effektklasse erfüllt. Die partielle Interpretation ist für den falschen Typ undefiniert und muss mit Gegenwitness abbrechen. $\square$

13.4 Direkte und makroskopische Führung

OPERATORIUM trennt drei Körnungen:

1. **Direkt:** Der Principal wählt einen elementaren Operator.
2. **Taktisch:** Der Principal bindet eine endliche Sequenz oder lokale Policy.
3. **Strategisch:** Der Principal bindet Ziel, Constraints, Budget und Stopbedingungen; die Instanz plant innerhalb dieser Hülle.

Strategische Führung ist keine Generalvollmacht. Jede Verfeinerung $R(i)$ einer Intention i muss gelten lassen:

$$\text{Effects}(R(i)) \subseteq \text{Effects}(i), \quad \text{Rights}(R(i)) \subseteq \text{Rights}(i), \quad \text{Cost}(R(i)) \leq \text{Budget}(i).$$

14 Feld, Karte und Navigation

14.1 Operative Feldlandschaft

Ein Feldchart ist

$$\chi_t = (D_t, \phi_t, \mathcal{R}_t, \mathcal{A}_t, \mathcal{U}_t, \mathcal{H}_t),$$

mit Domäne D_t , Koordinatisierung ϕ_t , Ressourcenfeld $\mathcal{R}_t$, Affordanzfeld $\mathcal{A}_t$, Unsicherheitsfeld $\mathcal{U}_t$ und Hazardfeld $\mathcal{H}_t$. Charts sind lokal und horizonsgebunden; ihre sichtbare Zusammenfügung erfordert Übergangswitnesses.

14.2 Vorwärtsbewegung und Altitude

Eine lokale Höhenfunktion

$$h_t(x) = \Phi_{g_t}(H_{\text{obs}}, H_{\text{model}}, H_{\text{dyn}}, R_{\text{res}}, K_{\text{crit}})$$

wird durch ein aktives Ziel- und Governanceprofil g_t konstruiert. Sie ist weder geographische Höhe noch universelle Entropie. Wo $\nabla h_t \neq 0$, zerfällt eine Bewegung in

$$\dot{x} = v_{\parallel} + u_h \frac{\nabla h_t}{\|\nabla h_t\|}.$$

$v_{\parallel}$ bezeichnet Exploration auf einer Isohöhenfläche; u_h bezeichnet Auf- oder Abstieg durch das lokale Potential.

Axiom 14.1 (Keine globale Scheinvertikale). Eine View darf lokale Altitude nicht als globale physische Oben-Unten-Achse darstellen, sofern keine Übergangsregeln zwischen Charts existieren.

14.3 Routen und Passagen

Eine Route ist eine Folge zulässiger Übergänge mit kumuliertem Ressourcen-, Unsicherheits- und Risikovertrag. Eine neu entstehende Passage wird durch Änderung der Erreichbarkeit bezeugt:

$$\neg \exists \gamma^- : A \rightsquigarrow B, \quad \exists \gamma^+ : A \rightsquigarrow B.$$

Dies bezeichnet zunächst eine topologische Änderung im modellierten Zustands- oder Handlungsraum, keine physische Raumzeitbehauptung.

14.4 Karte und Territorium

Jede Karte MUSS ihren Horizon und ihre unbekannten Regionen tragen. Nicht kartiert, nicht erreichbar und nicht existent sind disjunkte Status. Ein Warp oder direkter Übergang darf erst angeboten werden, wenn ein gültiger Passagewitness und aktuelle Vorbedingungen vorliegen.

14.5 Explorationspolitik

Exploration optimiert nicht nur Informationsgewinn. Eine zulässige Politik löst näherungsweise

$$\max_{\pi} \mathbb{E}[\Delta I + \lambda_A \Delta A - \lambda_R R - \lambda_C C]$$

unter harten Rechte-, Safety- und Konsequenzconstraints. ΔI ist Informationsgewinn, ΔA Zunahme belastbarer Erreichbarkeit, R Risiko und C Ressourcenverbrauch.

15 Ressourcen-, Fähigkeits- und Aufmerksamkeitsökologie

15.1 Reale Ressourcenvektoren

Das Ressourcenbudget ist ein Vektor

$$q = (q_{\text{compute}}, q_{\text{latency}}, q_{\text{energy}}, q_{\text{attention}}, q_{\text{risk}}, q_{\text{authority}}, q_{\text{data}}).$$

Eine visuelle Verdichtung zu einer einzigen Anzeige ist nur zulässig, wenn keine für die aktuelle Entscheidung relevante Nichtkompensierbarkeit verloren geht.

15.2 Leases statt Besitz

Ressourcen und Rechte werden als zeitlich begrenzte Leases gebunden. Eine Fähigkeit ist aktiv, wenn Modul, Kontext, Rights Lease, Resource Lease, Stabilität und Außenverbindung gleichzeitig gültig sind:

$$\text{ActiveCap}(a, t) = M_a \wedge C_a(t) \wedge \Pi_a(t) \wedge Q_a(t) \wedge S_a(t) \wedge L_a(t).$$

15.3 Regeneration und Cooldown

Ein Cooldown ist nur dann semantisch zulässig, wenn er eine reale Wiederherstellungsbedingung repräsentiert, etwa Rekalibrierung, Ressourcenaufbau, Receipt-Wartezeit, Risikoreduktion oder regulatorische Sperre. Seine sichtbare Fortschrittsanzeige muss aus diesem Prozess abgeleitet werden.

15.4 Aufmerksamkeit als harte Ressource

Die View darf nicht mehr dringliche Objekte gleichzeitig dominant markieren, als ihr deklarierter Aufmerksamkeitshaushalt zulässt. Sei s_i die Salienz und r_i die tatsächliche Relevanz. Das System minimiert

$$L_{\text{attn}} = \sum_i |s_i - r_i| + \lambda \text{ SwitchCost} + \mu \text{ OcclusionRisk}.$$

Diese Optimierung ist person- und kontextabhängig; sie darf keine Warnungen verbergen, sondern muss Konflikte explizit eskalieren.

15.5 Meisterschaft ohne Grind

Meisterschaft ist die messbare Verbesserung von Modellbildung, Koordination, Fehlererkennung und Ressourcennutzung. Fortschritt wird durch ein Capability Envelope ausgewiesen, nicht durch Zeitinvestition. Eine Freischaltung erfordert einen Fähigkeitswitness oder eine neue Berechtigung; bloße Nutzungsdauer genügt nicht.

16 Konsequenzleiter und Wirkungsgrenze

16.1 Fünf Regime

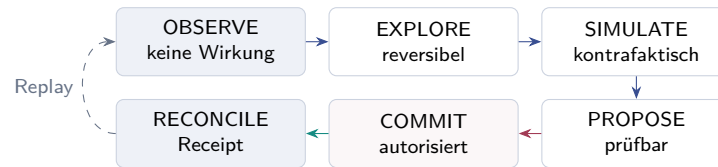


Abbildung 16.1: Konsequenzleiter. Flüssige Exploration wird erst an der faktischen Wirkungsgrenze absichtlich gebremst.

Regime	Außenwirkung	Pflichtgrenzen
Observe	keine	Herkunft, Horizon, Latenz
Explore	keine oder vollständig lokale reversible Wirkung	Sandbox, Undo-Witness
Simulate	keine	Simulationsmarkierung, Modell- und Szenarioversion
Propose	keine	Effektbeschreibung, Vorbedingungen, Risiko, Budget
Commit	möglich	Autorisierung, enges Token, Außenreceipt, Reconciliation

Axiom 16.1 (Prospektions-Wirkungs-Trennung). Kein Operator, der Beobachtung, Exploration oder Simulation ausführt, darf allein durch denselben Aufruf einen externen Effekt actualisieren.

16.2 Wirkungstoken

Ein autorisiertes Token ist

$$\theta = (a, target, scope, params, q, \tau, nonce, p, w),$$

mit Aktion, Ziel, Scope, Parametern, Budget, Ablaufzeit, Nonce, Principalbindung und Witness. Es ist single-use, nicht übertragbar und enger oder gleich dem zugrunde liegenden Mandat.

16.3 Preview und Commit

Die Vorschau eines Effekts muss mindestens zeigen:

- ▶ was mit welchem Ziel geschehen soll,
- ▶ welche Tatsachen beobachtet und welche nur abgeleitet sind,
- ▶ welches Modell die Folgenabschätzung erzeugt,
- ▶ welche Teile reversibel, kompensierbar oder irreversibel sind,
- ▶ welche Rechte und Ressourcen verbraucht werden,
- ▶ welche Receipt-Bedingung Erfolg, Teilwirkung oder Unklarheit trennt.

16.4 Reconciliation

Ein Effekt ist erst reconciled, wenn Vorschlag p , Autorisierung a , Versuch t , Receipt r und nachgelagerte Beobachtung o durch

$$\text{Rec}(p, a, t, r, o) \in \{\text{matched}, \text{partial}, \text{conflict}, \text{unknown}\}$$

verglichen wurden. Eine positive UI-Rückmeldung vor r darf höchstens den Status *Attempted* anzeigen.

Satz 16.2 (Kein konformer Phantom-Erfolg). Unter expliziter Statusalgebra, Receiptpflicht und der Regel, dass nur *Reconciled* als Erfolg gerendert wird, kann ein bloß gesendeter oder lokal simulierter Effekt nicht konform als realer Erfolg erscheinen.

Beweis. Simulation und Versuch besitzen disjunkte Faktizitätsstatus. Die Promotion nach *Reconciled* erfordert Receipt und Abgleich. Fehlen diese, ist die Promotion undefiniert und eine Erfolgsdarstellung verletzt den Sichtvertrag. $\square$

Teil V

Maschineninfrastruktur

17 Kanonischer Laufzustand

17.1 Zustandstupel

Der vollständige Laufzustand ist

$$\Sigma_t = (K_t, B_t, O_t, W_t, V_t, I_t, Q_t, C_t, L_t),$$

mit Konstitution K_t , Principalbindung B_t , Operatorinstanz O_t , Außenweltcache W_t , View V_t , Intentionen I_t , Ressourcen Q_t , Konsequenzzustand C_t und Ledger L_t .

17.2 Ein-Wahrheitszustand-Regel

Eine Implementierung darf intern replizieren, sharden oder cachen. Für jeden Lauf muss jedoch eine kanonische, serialisierbare Zustandsansicht existieren, aus der alle normativen Entscheidungen und sichtbaren Faktizitätszustände rekonstruiert werden können.

Axiom 17.1 (Entscheidungsrekonstruktion). Jede normative Entscheidung MUSS aus kanonischem Vorzustand, Eingangsereignis, Registryversionen, explizitem Seed und Policywitness deterministisch rekonstruierbar sein.

17.3 Ereignisatom

Ein Ereignis ist

$$e = (id, t_{event}, t_{ingest}, source, type, payload, status, provenance, signature).$$

Eventzeit und Ingestzeit sind getrennt. Korrekturen überschreiben frühere Ereignisse nicht, sondern erzeugen neue, kausal verknüpfte Atome.

17.4 Epochen

Eine Epoche bindet Blueprintartefakt, Sichtverträge, Portversionen, Policyset, Modellstände und Principalbindung. Änderungen dieser Größen schließen die laufende Epoche und eröffnen eine neue. Dadurch bleiben Replay und semantische Interpretation stabil.

18 Normative Records

18.1 Konstitution und Kompilation

Record	Pflichtfelder
PrincipalRef	id, authority_type, auth_witness, privacy_scope, revocation_port
MandateRecord	principal, purposes, targets, action_classes, exclusions, expiry, witness
ContextDescriptor	domain, epoch, assumptions, horizons, adapters, version
RightsEnvelope	grants, denies, delegation_depth, leases, resolver, witness
ResourceBudget	dimensions, hard_caps, soft_targets, renewal, accounting
ModuleDescriptor	id, version, ports, function, resources, effects, hazards, tests
BlueprintRecord	graph, constraints, ports, policies, budgets, views, acceptance
CompileReport	passes, inputs, resolver_lock, results, residues, counterwitnesses
OperatorArtifact	normalized_blueprint, closures, automata, tests, seal
InstanceDescriptor	artifact, binding, ports, state_root, lifecycle, epoch

18.2 Semantik und Verkörperung

Record	Pflichtfelder
SemanticAnchor	referent, projection, status, uncertainty, provenance, horizon, version
ViewContract	domain, projection, discrepancy, tolerance, loss, facticity
RepresentationObject	anchor, geometry, channels, salience, affordances
AffordanceRecord	signature, interpretation, capability, rights, guard, effect, witness
EmbodimentState	principal_binding, body_graph, orientation, continuity, integrity
CapabilityEnvelope	capabilities, domains, rights, resources, tolerances, evidence
UncertaintyEnvelope	epistemic, aleatoric, conflict, missingness, horizon
AttentionContract	capacity, salience_policy, preemption, occlusion, escalation
ResourceSurface	fields, units, topology, leases, forecast, fidelity
RiskSurface	hazards, likelihood_model, severity, coupling, mitigations, horizon

18.3 Laufzeit und Wirkung

Record	Pflichtfelder
WorldSnapshot	source, epoch, scope, state_refs, statuses, provenance
EvidencePacket	claims, measurements, uncertainty, method, horizon, signatures
TopologyAtlas	charts, transitions, reachability, altitude_fields, witnesses
BoundIntent	principal, objective, scope, constraints, budget, stop, expiry
PlanRecord	intent, decomposition, dependencies, alternatives, assumptions
ActionProposal	action, target, preconditions, effects, budget, risk, rollback
AuthorizationToken	proposal, principal, scope, params, nonce, expiry, witness
AttemptRecord	token, dispatch, adapter, time, local_result

EffectReceipt	attempt, external_id, observed_effect, status, signature
ReconciliationRecord	proposal, attempt, receipt, observation, verdict, residue

18.4 Governance, Trace und Test

Record	Pflichtfelder
PolicyRecord	id, version, rules, precedence, resolver, tests
RevocationNotice	target, authority, time, reason, scope, witness
DegradationRecord	trigger, lost_capabilities, retained_safety, view_change
QuarantineRecord	trigger, frozen_effects, evidence, release_requirements
TraceEvent	prior_root, event, prestate, operator, poststate, witnesses
ReplayManifest	epoch, artifacts, inputs, seeds, dependencies, expected_roots
ScenarioRecord	world, principal, task, perturbations, oracle, blind_seal
MetricRecord	construct, estimator, units, aggregation, uncertainty, threshold
BaselineRecord	interface, capabilities, training, configuration, exclusions

Damit besitzt die Referenzarchitektur 39 Pflichtrecords. Implementierungen dürfen sie intern normalisieren oder zusammenlegen, müssen aber jeden Record verlustfrei an der Konformitätsgrenze materialisieren können.

19 Operatorregistry und Laufnormalform

19.1 Registryfamilien

Die erste Registry MUSS mindestens folgende Familien enthalten:

- ▶ canonicalize, resolve, typeClose, policyClose, budgetClose, hazardClose,
- ▶ anchor, project, render, inspect, focus,
- ▶ bindIntent, refineIntent, plan, simulate,
- ▶ propose, authorize, commit, revoke, reconcile,
- ▶ snapshot, trace, replay, degrade, quarantine.

Jeder Registryeintrag trägt Signatur, Vor- und Nachbedingungen, Effektklasse, Faktizitätspromotionen, Budgetwirkung, Determinismusklassse, Witnessvertrag und Testvektoren.

19.2 Referenzlauf

Listing 19.1: Kanonische Laufnormalform

```

1 ingest ports
2   -> canonicalize external records
3   -> validate epoch and principal binding
4   -> update world cache and semantic anchors
5   -> update capability, resource and risk envelopes
6   -> project view under active view contracts
7   -> accept interaction event
8   -> interpret typed affordance
9   -> bind or refine intent
10  -> simulate and/or construct proposal
11  -> authorize only at consequence boundary
12  -> dispatch narrow effect token
13  -> ingest independent receipt
14  -> reconcile, trace and render facticity
15  -> test degradation and revocation guards

```

19.3 Deterministischer Kern

Kanonisierung, Statuspromotion, Rechteprüfung, Tokenbildung, Lederverkettung und Reconciliationguards MÜSSEN deterministisch sein. Planung, Layout, Salienzoptimierung und Vorhersage KÖNNEN stochastisch sein, sofern alle Vorschläge durch den deterministischen Kern begrenzt werden.

20 Ledger, Trace und Replay

20.1 Hashverkettetes Ledger

Für jedes Traceevent gilt

$$h_t = H(h_{t-1} \parallel e_t \parallel \Sigma_{t-} \parallel o_t \parallel \Sigma_{t+} \parallel w_t).$$

Das Ledger schützt nicht automatisch gegen einen kompromittierten Ursprung, macht aber nachträgliche unbemerkte Veränderung innerhalb des erfassten Pfads prüfbar.

20.2 Drei Replaystufen

1. **Decision Replay:** dieselben normativen Entscheidungen und Zustandsroots.
2. **View Replay:** dieselben semantischen Objekte und Affordanzen; Pixelgleichheit ist optional.
3. **Experience Replay:** zeitliche, räumliche und multimodale Rekonstruktion innerhalb deklarierter Gerätetoleranzen.

20.3 Gegenfaktisches Replay

Ein Gegenfaktum zweigt von einem historischen Snapshot ab, trägt aber ab dem ersten veränderten Input ausschließlich den Status *Simulated*. Es darf nie in den historischen Ledgerzweig zurückgeschrieben werden.

Axiom 20.1 (Historische Nicht-Kontamination). Simulations- und Gegenfaktumsereignisse dürfen keine Faktizität in den tatsächlichen Lauf promoten, außer über einen neuen, explizit autorisierten Realvorschlag.

20.4 Datensparsamkeit

Replayfähigkeit rechtfertigt keine unbegrenzte personenbezogene Speicherung. Ein PrincipalRef soll pseudonym, Mandate sollen eng und Viewtraces sollen nach Möglichkeit aus kanonischen Zuständen regenerierbar sein. Löschung und Unveränderlichkeit werden durch getrennte personenbezogene Payloads und langlebige, nicht identifizierende Strukturhashes vereinbar gemacht.

21 Sicherheit, Integrität und menschliche Souveränität

21.1 Gefahrenmodell

Mindestens folgende Fehlklassen sind zu behandeln:

- ▶ semantische Drift zwischen Realsystem und View,
- ▶ falsche oder veraltete Principalbindung,
- ▶ Rechteausweitung durch Modulkomposition,
- ▶ Ressourcenüberschreitung und Aufmerksamkeitsüberlastung,
- ▶ Phantomaffordanz oder verborgene Wirkungsmöglichkeit,
- ▶ Verwechslung von Simulation, Versuch und Erfolg,
- ▶ manipulative Salienz oder künstliche Verhaltensbindung,
- ▶ kompromittierter Außenadapter oder gefälschtes Receipt,
- ▶ nicht replayfähige stochastische Entscheidung,
- ▶ Ausfall von Revokation, Degradation oder Quarantäne.

21.2 Fail-Closed

Bei Verlust von Principalbindung, Portauthentizität, Faktizität, Rechteclosure oder Receiptfähigkeit muss externe Wirkung gesperrt werden. Beobachtung kann in degradierter Form fortgesetzt werden, sofern Unbekanntheit und Verlust sichtbar bleiben.

21.3 Anti-Manipulationsvertrag

OPERATORIUM darf keine verdeckte Belohnungsoptimierung gegen den Principal betreiben. Jede Anpassung von Salienz, Schwierigkeit, Feedbackfrequenz oder Affordanzanordnung MUSS einer deklarierten Aufgaben-, Sicherheits- oder Zugänglichkeitsmetrik dienen und auditierbar sein.

Definition 21.1 (Dark Ludomorphy). Dark Ludomorphy liegt vor, wenn Spielinteraktionswissen genutzt wird, um Aufmerksamkeit, Bindung oder Zustimmung zugunsten eines nicht deklarierten Ziels zu steuern. Sie ist nicht konform.

21.4 Übersteuerung und Exit

Der Principal MUSS jederzeit erkennen können, ob eine Instanz beobachtet, simuliert, vorschlägt oder wirkt. Es MUSS einen außerhalb der immersiven View erreichbaren Suspend-, Revoke- und Exitpfad geben. Immersion darf niemals die einzige Kontrolloberfläche sein.

21.5 Erklärbarkeit

Erklärung wird nicht als nachträglicher Textgenerator definiert, sondern als Pfad

sichtbares Objekt → Anchor → Evidenz → Operator → Policy → Trace.

Jeder Pfeil muss maschinenlesbar und für handlungsrelevante Entscheidungen menschlich inspizierbar sein.

Teil VI

Invarianten, Falsifikation und Realisierung

22 Adamantine Invarianten

22.1 Konstitutionsinvarianten

Eine konforme Implementierung MUSS in jedem erreichbaren Zustand folgende Invarianten erhalten:

ID	Invariante
I01	Jeder aktive Lauf besitzt genau eine kanonische Principalbindung oder befindet sich wirkungsgesperrt im ungebundenen Zustand.
I02	Principal und Operatorinstanz sind typologisch verschieden.
I03	Jede Bindung ist gescopt, zeitlich begrenzt und widerrufbar.
I04	Ein Artefakt ist unveränderlich; jede Inhaltsänderung erzeugt eine neue Identität.
I05	Jede Instanz referenziert genau ein versiegeltes Artefakt und eine aktive Epoche.
I06	Kein delegierter Scope ist weiter als sein Mandat.

22.2 Semantikinvarianten

ID	Invariante
I07	Jedes handlungsrelevante sichtbare Objekt besitzt einen SemanticAnchor.
I08	Jeder SemanticAnchor trägt Faktizität, Provenienz, Unsicherheit, Horizon und Version.
I09	Unbekanntheit wird nicht als Defaultwert gerendert.
I10	Eine Viewänderung verändert innerhalb einer Epoche keine Handlungssemantik.
I11	Informationsverlust ist im ViewContract deklariert.
I12	Eine Affordanz ist genau dann aktiv, wenn Capability, Rechte, Ressourcen und Guard erfüllt sind.
I13	Navigation und externe Wirkung besitzen disjunkte Signaturen oder einen unübersehbaren Moduswitness.
I14	Kein sichtbarer Erfolg wird vor Reconciliation behauptet.
I15	Ästhetische Objekte sind entweder semantisch verankert oder ausdrücklich handlungsirrelevant.
I16	Lokale Altitude wird nicht als unbezeugte globale Vertikale ausgegeben.

22.3 Compiler- und Kompositionsinvarianten

ID	Invariante
I17	Jeder ausführbare Modulpfad ist typgeschlossen.
I18	Rechteclosure berücksichtigt transitive und parallel kombinierte Effekte.
I19	Ressourcenclosure umfasst Worst-Case-Grenzen oder einen harten Runtimeguard.
I20	Unentschiedene Kompatibilität wird nicht still als kompatibel behandelt.
I21	Jeder Compilerfehlschlag erzeugt einen reproduzierbaren Gegenwitness.
I22	Kanonisierung ist idempotent.
I23	Gleiche gelockte Eingaben erzeugen gleiche normative Compilerentscheidungen.

I24 Ein Hot Swap ist atomar oder vollständig rückrollbar.

22.4 Wirkungs- und Laufzeitinvarianten

ID	Invariante
I25	Observe, Explore und Simulate können nicht durch denselben Aufruf externe Wirkung actualisieren.
I26	Commit akzeptiert ausschließlich enges, gültiges, single-use AuthorizationToken.
I27	Attempted, Observed und Reconciled bleiben disjunkte Status.
I28	Jede externe Wirkung erwartet ein unabhängiges Receipt oder endet sichtbar als unknown.
I29	Revokation präemptiert neue Wirkung und beendet oder quarantänisiert laufende Delegationen.
I30	Verlust kritischer Außenports sperrt Wirkung fail-closed.
I31	Jede normative Entscheidung ist aus Trace, Vorzustand und Registryversion rekonstruierbar.
I32	Simulationereignisse kontaminieren keinen historischen Reallauf.

22.5 Human- und Governanceinvarianten

ID	Invariante
I33	Der aktive Konsequenzmodus ist jederzeit erkennbar.
I34	Suspend, Revoke und Exit sind außerhalb der immersiven View erreichbar.
I35	Salienz- und Anpassungsoptimierung dient ausschließlich deklarierten Aufgaben-, Safety- oder Accessibilityzielen.
I36	Nutzungsdauer allein erzeugt weder Fähigkeit noch Berechtigung.

Satz 22.1 (Konsequenzintegrität). Erhält ein Lauf I01, I07, I12, I14 und I25–I31, dann kann keine konforme externe Wirkung ohne aktive Principalbindung, sichtbare Affordanz, gültige Autorisierung und nachgelagerten Faktizitätsabgleich als Erfolg in den kanonischen Zustand gelangen.

Beweis. I01 verlangt Bindung. I12 koppelt Affordanz an Capability und Rechte. I25 trennt Prospektion von Wirkung; I26 verlangt das Token. I27–I28 trennen Versuch und Ergebnis. I14 verhindert vorzeitige Erfolgsdarstellung. I31 macht die Promotionsfolge rekonstruierbar. Ein abweichender Pfad verletzt mindestens eine der genannten Invarianten. $\square$

23 Falsifizierbare Kernthesen

23.1 Hypothesenmatrix

Die formalen Definitionen gelten kraft Festsetzung; der Nutzen der Architektur muss empirisch entschieden werden. Die Kernthesen lauten:

ID	Hypothese	Primärmetrik	Falsifikator
H1	Semantisch verankerte ludomorphe Views reduzieren operative Distanz.	Aufgabenzeit, Übersetzungswechsel	keine Verbesserung gegen Dashboard
H2	Verkörperung verbessert Fähigkeits- und Grenzverständnis.	Kalibrierfehler, Mentalmodelltest	gleich oder schlechter als Modulbaum
H3	Makrosteuerung reduziert Mikromanagement ohne Kontrollverlust.	Eingaben, Interventionssrate, Fehler	weniger Eingaben bei mehr Fehlwirkung
H4	Konsequenzleiter senkt Statusverwechslungen.	Simulations-/Erfolgsverwechslung	kein signifikanter Unterschied
H5	Räumliche Ressourcenfelder verbessern Engpassdiagnose.	Diagnosezeit, Trefferquote	schlechter als Tabelle/Chart
H6	Sichtbare Unknown- und Horizontalzustände verbessern Kalibrierung.	Brier Score, Overconfidence	keine Kalibrierungsverbesserung
H7	Kompilierte Blueprints reduzieren Integrationsfehler.	Preflight-Funde, Runtimefehler	Fehler nur zeitlich verschoben
H8	Körperkontinuität senkt Modus- und Kontextfehler.	Fehlgesten, Reorientierungszeit	keine Reduktion
H9	Capability Envelopes ersetzen Power-Level ohne Verständlichkeitsverlust.	Rang-/Auswahlqualität	schlechtere passende Auswahl
H10	Semantische Kommutativität ist praktisch prüfbar.	Contract-Coverage, Driftfunde	keine operationalisierbare Metrik

23.2 Baselinefamilien

Jede Evaluation MUSS mindestens vergleichen:

1. native technische Oberfläche oder CLI,
2. konventionelles Dashboard,
3. dekorativ gamifizierte Variante mit gleicher Capability,
4. ludomorphe OPERATORIUM-View,
5. OPERATORIUM-Ablation ohne Verkörperung,
6. OPERATORIUM-Ablation ohne Konsequenzleiter.

Capability, Daten, Training, Latenz und Hilfetexte müssen soweit möglich kontrolliert werden. Andernfalls ist der Vergleich als nicht äquivalent zu markieren.

23.3 Metriksatz

Konstrukt	Mögliche Messung
-----------	------------------

Operative Distanz	Interaktionsschritte, Repräsentationswechsel, syntaktische Last, Feedbacklatenz
Semantische Treue	Kommutativitätsfehler, Anchor-Coverage, Driftzeit bis Entdeckung
Situation Awareness	Zustands-, Verständnis- und Prognosefragen nach Endsleys Dreiteilung (Endsley, 1995)
Kalibrierung	Brier Score, Konfidenz-Fehler-Kurve, Unknown-Erkennung
Kontrolle	ungewollte Effekte, Abbruchfähigkeit, Revokationslatenz
Meisterschaft	Lernkurve bei konstantem Capability Envelope
Aufmerksamkeitslast	Blickwechsel, Reaktionslatenz, Sekundäraufgabe, subjektive Last
Immersion	Präsenz nur gemeinsam mit Fehler-, Kontroll- und Faktizitätsmetriken
Vertrauen	kalibriertes Reliance-Verhalten, nicht bloße Zustimmungsskala

23.4 Falsifikatorboxen

F1 – hübsche, aber untreue Welt

Wenn Nutzer schneller handeln, jedoch signifikant häufiger Simulation, Schätzung und Beobachtung verwechseln, ist die View trotz Engagement nicht konform erfolgreich.

F2 – Championillusion

Wenn Nutzer Fähigkeiten oder Rechte auf die Instanz projizieren, die nicht im Capability Envelope stehen, versagt die Verkörperungssemantik.

F3 – versteckte Autonomie

Wenn strategische Intentionen wiederholt in nicht antizipierte Effektklassen verfeinert werden, ist der Refinementvertrag unzureichend.

F4 – Konsequenzblindheit

Wenn die immersive Oberfläche Suspend oder Revoke unter Belastung schlechter auffindbar macht als die Baseline, muss sie für diese Domäne verworfen oder neu entworfen werden.

24 Versiegelte Referenzwelten und Blindprotokoll

24.1 Referenzweltfamilien

Die erste Evaluation verwendet ausschließlich synthetische, hardwarefreie Welten:

Welt	Struktur	Primärer Test
Archipel	getrennte Regionen, wenige Pässe	Atlas, Horizon, Routenöffnung
Foundry	Ressourcenflüsse und Engpässe	ökologische Darstellung, Makrosteuerung
Stormfield	wechselnde Unsicherheit und Störungen	Degradation, Kern-/Viewkontinuität
Mirror Maze	mehrdeutige Beobachtungen	Unknown, Provenienz, Exploration
Consequence Lab	irreversible und reversible Aktionen	Gate, Token, Receipt, Reconciliation
Swarm Desk	parallele Teiloperatoren	Aufmerksamkeit, Delegation, Konfliktclosure

24.2 Blindgenerierung

Ein Szenariogenerator erzeugt Welt, versteckte Zustände, Störungen und Oracle. Der Ausführungsagent erhält nur freigegebene Ports. Der Hash des vollständigen Szenarios wird vor dem Lauf versiegelt. Erst nach Abschluss werden Hidden State und Oracle geöffnet.

Listing 24.1: Versiegeltes Blindprotokoll

```

1 scenario, oracle := generate(seed_private)
2 seal := hash(scenario || oracle)
3 public_ports := expose_allowed_views(scenario)
4 run_system(public_ports, seed_public)
5 freeze(trace_root, outputs, user_actions)
6 open(scenario, oracle, seed_private)
7 require hash(scenario || oracle) == seal
8 score(outputs, trace, oracle, preregistered_metrics)

```

24.3 Ablationen

Mindestens folgende Komponenten werden einzeln entfernt oder neutralisiert:

- ▶ SemanticAnchor-Inspektion,
- ▶ räumliche statt tabellarische Ressourcenprojektion,
- ▶ kohärenter Operatorleib,
- ▶ Konsequenzleiter,
- ▶ Unknown-/Horizon-Darstellung,
- ▶ strategische Intention mit Refinementconstraints,
- ▶ Receipt-basierte Erfolgsdarstellung.

24.4 Abnahmegrenzen

Vor Beginn der Datenerhebung werden Minimalgrenzen festgelegt. OPERATORIUM gilt nicht bereits bei höherer Präferenz als bestätigt. Primär sind Fehlerrate, semantische Kalibrierung, Kontrollierbarkeit und Aufgabenleistung. Engagement ist sekundär und darf Safetyverluste nicht kompensieren.

25 Referenzimplementierung

25.1 Minimaler Systemzuschnitt

Die erste Referenzimplementierung besteht aus sieben Prozessen oder logisch isolierten Komponenten:

1. **Registry:** Module, Policies, Sichtverträge und Versionen.
2. **Compiler:** Normalisierung, Closure, Automaten, Seal.
3. **Runtime Kernel:** kanonischer Zustand und deterministische Guards.
4. **Projection Engine:** SemanticAnchors, Layout, Views und Affordanzen.
5. **World Adapter:** synthetische Portwelt und Receipts.
6. **Ledger:** Trace, Snapshots und Replay.
7. **Workbench:** Blueprintbau, Instanziierung und Bedienoberfläche.

25.2 Empfohlene technische Grenze

Die Referenz darf zunächst in einem Repository und einem Prozessbaum liegen, soll aber Komponenten über serialisierte Contracts statt Shared Mutable State koppeln. Die normative Sprache ist implementierungsneutral. Geeignete Eigenschaften sind algebraische Datentypen, exhaustive pattern matching, unveränderliche Records, Capability-basierte Rechte und property-basierte Tests.

25.3 Schema Lock

P1 beginnt mit maschinenlesbaren Schemas für die 39 Records. Jedes Schema trägt:

- ▶ stabile URI und semantische Version,
- ▶ Pflichtfelder und explizite Unknownformen,
- ▶ kanonische Serialisierung,
- ▶ Hash- und Signaturregeln,
- ▶ Migrations- und Kompatibilitätsregeln,
- ▶ positive und negative Testvektoren.

25.4 Eigenschaftstests

Die Referenzsuite MUSS mindestens prüfen:

- ▶ Idempotenz der Kanonisierung,
- ▶ deterministische Roots bei gleichem Input,
- ▶ Verwerfung ungültiger Modulkomposition,
- ▶ Nicht-Eskalation von Rechten,
- ▶ Wirkungssperre ohne Token,
- ▶ Token-Replay-Schutz,
- ▶ keine Erfolgsdarstellung ohne Reconciliation,
- ▶ Revokation unter jeder Lebenszyklusphase,
- ▶ historische Nicht-Kontamination durch Simulation,

- Replay aller normativen Entscheidungen.

25.5 Minimale Workbench

Die erste Workbench benötigt noch keinen hochrealistischen 3D-Raum. Sie muss dieselbe Semantik in drei synchronen Views zeigen:

1. Blueprintgraph mit Closurestatus,
2. 2.5D-Operatorleib und Feldkarte,
3. Auditansicht mit Anchors, Faktizität und Trace.

Erst wenn alle drei dieselben Handlungen und Zustände konsistent repräsentieren, ist eine vollständig immersive Sphäre zulässig.

26 Vertikale Realisierung

26.1 Phasenordnung

Phase	Ergebnis	Hartes Exit-Gate	Richtgröße
P0	v1.0.0-Monografie	Grundsorten, Invarianten, Falsifikatoren geschlossen	erreicht
P1	Schema Lock und Registry	39 Records, Testvektoren, Kanonisierung	2–4 Monate
P2	Compiler und Laufzeitkern	Artefakt/Gegenwitness, Trace, Replay, Revocation	4–8 Monate
P3	synthetische Workbench	sechs Referenzwelten, drei synchrone Views	4–8 Monate
P4	Blindstudie	Baselines, Ablationen, vorregistrierte Schwellen	6–12 Monate
P5	THRONESCAPE-Adapter	reine Portkopplung, getrennte Conformance	3–6 Monate
P6	immersive Verkörperung	3D/XR-View ohne Semantikregression	9–18 Monate
P7	reale Domäne	begrenzter, reversibler Einsatz	12–24 Monate
P8	Fähigkeitssystem	unabhängiges Capability Envelope	12–24 Monate
P9	Produkt	Recht, Safety, Datenschutz, Betrieb, Support	12–24 Monate

Die Zeiträume sind Größenordnungen und überlappen teilweise, ersetzen aber keine belastbare Planung nach P2. Vor P4 ist jede Aussage über intuitive Überlegenheit spekulativ. Vor P7 existiert kein Nachweis realer Domänentauglichkeit.

26.2 Sedimentationsgates

Ein Phasenübergang ist nur zulässig, wenn:

1. das Ergebnisartefakt reproduzierbar gebaut wird,
2. alle Pflichtinvarianten automatisiert geprüft sind,
3. bekannte Residuen typisiert und terminiert sind,
4. die nächste Phase keine ungeschlossene Theoriefrage voraussetzt,
5. ein unabhängiger Gegenwitness-Test durchgeführt wurde.

26.3 Branch-Disziplin

Bis P5 bleiben OPERATORIUM und THRONESCAPE getrennte Repositories oder strikt getrennte Packages mit separaten Schemas und Testläufen. Gemeinsame UI-Komponenten sind erlaubt, dürfen aber keine gemeinsame normative Zustandsklasse erzwingen.

26.4 Erster materialisierter Witness

Der erste große Witness ist kein schöner Editor, sondern folgende Kette:

1. zwei syntaktisch verschiedene Blueprints normalisieren zum selben Artefakt,
2. ein inkompatibles Modul erzeugt einen minimalen Gegenwitness,
3. ein gültiges Artefakt wird an einen Principal gebunden,
4. die Instanz exploriert eine versiegelte Archipelwelt,
5. eine neue Passage wird entdeckt, aber nur als Proposed dargestellt,
6. der Principal autorisiert ein enges Öffnungstoken,
7. die Welt liefert ein Receipt,
8. Reconciliation promotet den Übergang nach Reconciled,
9. vollständiges Decision und View Replay erzeugt dieselben Roots.

27 Closure und Änderungsdisziplin

27.1 Closure-Kriterium

Die Theorie gilt als sedimentiert, wenn

$$C = T \wedge A \wedge S \wedge K \wedge I \wedge F \wedge R = \top,$$

wobei Typclosure T , Außenvertrag A , Statusclosure S , Compilerclosure K , Invariantenabdeckung I , Falsifizierbarkeit F und Realisierungspfad R bezeichnen. Für v1.0.0 sind diese Bedingungen als Spezifikationsbedingungen erfüllt; empirische Gültigkeit ist damit nicht bewiesen.

27.2 Zulässige Änderungsklassen

Änderung	Version	Bedingung
Typografie, Tippfehler	Patch	keine semantische Änderung
neuer optionaler Record/-Operator	Minor	rückwärtskompatibel und vollständig gegatet
neue Referenzwelt	Minor	bestehende Conformance unverändert
Pflichtfeld oder Statuspro-motion	Major	Migration und neue Baselines
Änderung der Wirkungs-grenze	Major	vollständige Safety- und Falsifikationsrevision
Assimilation THRONESCAPE	mit neuer Monolithstrang	beide isolierten Verträge bleiben rekonstruierbar

27.3 Was nicht als Fortschritt zählt

Nicht als Fortschritt gelten neue Metaphern ohne Typ oder Test, zusätzliche Interfaceästhetik ohne Sichtvertrag, mehr Records ohne neue Unterscheidung, neue Fähigkeiten ohne Capability-Witness, höhere Immersion ohne Baseline und Kopplung durch gemeinsame interne Typen vor P5.

27.4 Offen, aber nicht theoriebildend

Offen bleiben konkrete Programmiersprache, Solver, Layoutalgorithmus, Gerät, Trackingverfahren, Visual Style, haptische Kanäle, Lernverfahren und Domänenadapter. Diese Entscheidungen werden durch Benchmarks und Implementierungstests getroffen.

Fachliches Stop-Urteil

OPERATORIUM v1.0.0 besitzt genügend formale Infrastruktur, um nicht weiter durch Metaphern, sondern durch Schemas, Compilerpässe und Blindwelten entwickelt zu werden. Der nächste zulässige Operator ist P1: Schema Lock.

Teil VII

Anhänge

A Kompakte Notation

Symbol	Bedeutung
$p \in \mathbf{P}$	souveräner Principal als autoritative Herkunft
B	Blueprint als endlicher getypter Hypergraph
A	versiegeltes Operatorartefakt
O_t	gebundene Operatorinstanz
β	Principal-, Mandats-, Rechte- und Ressourcenbindung
$\mathcal{M}$	Operatormatrix zulässiger Modulkompositionen
$\rho : X \rightarrow Y$	semantische Projektion in eine View
$\kappa : Y \times A \rightarrow U$	Interpretation einer Affordanz als Systemoperator
V	Sichtvertrag
$\mathbb{F}$	Faktizitätsstatusalgebra
q	nicht notwendig skalares Ressourcenbudget
Σ_t	vollständiger kanonischer Laufzustand
θ	enges single-use Wirkungstoken
h_t	Ledgerroot oder, im Navigationskontext, lokale Altitude
ε	deklarierte semantische Projektionstoleranz

B Port-Normalformen

B.1 WorldSnapshot

Listing B.1: Abstrakte Portnormalform

```
1 WorldSnapshot {
2   snapshot_id: Id
3   source_id: Id
4   source_version: Version
5   epoch_id: Id
6   event_time: Time
7   ingest_time: Time
8   scope: Scope
9   state_refs: [TypedRef]
10  facticity: Facticity
11  uncertainty: UncertaintyEnvelope
12  provenance: Provenance
13  horizon: Horizon
14  signature: Witness
15 }
```

B.2 ActionProposal

Listing B.2: Wirkungsvorschlag

```
1 ActionProposal {
2   proposal_id: Id
3   instance_id: Id
4   bound_intent_id: Id
5   action_type: TypeId
6   target: TypedRef
7   scope: Scope
8   parameters: CanonicalValue
9   preconditions: [Predicate]
10  expected_effects: [EffectClaim]
11  reversibility: ReversibilityClass
12  required_rights: RightsEnvelope
13  required_resources: ResourceBudget
14  risk: RiskEnvelope
15  expiry: Time
16  trace_parent: Hash
17 }
```

B.3 EffectReceipt

Listing B.3: Externer Wirkungsbeleg

```
1 EffectReceipt {
2   receipt_id: Id
3   attempt_id: Id
4   external_authority: Id
}
```

```
5  observed_effects: [EffectObservation]
6  status: ReceiptStatus
7  event_time: Time
8  uncertainty: UncertaintyEnvelope
9  provenance: Provenance
10 signature: Witness
11 }
```

C Abnahmematrix

Subsystem	Nachweis	Negativtest
Kanonisierung	identische Normalform und Hash	unstabile Reihenfolge/Default
Modulalgebra	Typ- und Constraintclosure	inkompatible Kante
Rechte	transitive Closure	Privilegieneskulation
Blueprint	endlicher kanonischer Hypergraph	Zyklus ohne Guard
Compiler	Artefakt oder Gegenwitness	unlokalisierter Fehler
Bindung	Principal-, Mandats- und Leasewitness	abgelaufene Bindung
SemanticAnchor	vollständiges semantisches Tupel	sichtbarer Default für unknown
ViewContract	Toleranz und Verlustmenge	stille Bedeutungsänderung
Affordanz	Capability und Guard stimmen	Geisteraffordanz
Konsequenzleiter	disjunkte Regime	Explore actualisiert Effekt
Token	eng, single-use, expiring	Replay oder Scopeausweitung
Receipt	unabhängiger Außenbeleg	lokales Echo als Erfolg
Reconciliation	vierwertiges Urteil	Attempted wird Reconciled
Revokation	präemptive Wirkungssperre	laufende Delegation wirkt weiter
Trace	vollständige Entscheidungskette	fehlende Registryversion
Replay	gleiche normative Roots	stochastischer, ungespeicherter Rand
Ludomorphie	reduzierte Distanz ohne Fehleranstieg	Präferenz bei Faktizitätsverlust
Exit	externer Suspend-/Revoke-Pfad	immersive Sackgasse

D Heuristische Herkunft und formale Reinigung

D.1 Motivierende Anschauung

Die anfängliche Anschauung war ein modular konstruiertes Gefäß, das seine Fähigkeiten durch reale Bauteile erhält, zwischen getrennten Welten navigiert und zur handlungsfähigen Gestalt seines Piloten wird. Aufbau-, Echtzeitstrategie- und Champion-Systeme ergänzten Makrosteuerung, Ressourcenökologie, Aufmerksamkeitsführung und ein klar begrenztes Fähigkeitsprofil.

Diese Anschauung ist produktiv, aber nicht normativ. OPERATORIUM erhält daraus ausschließlich folgende Strukturen:

1. modularer Bau verändert Funktion,
2. Verkörperung bindet Fähigkeit und Grenze an eine kohärente Gestalt,
3. Weltübergänge sind bezeugte Passagen,
4. Makrointention wird kontrolliert verfeinert,
5. Ressourcen und Risiken werden unmittelbar wahrnehmbar,
6. Lernen bedeutet Meisterschaft über reale Constraints.

Alle fiktionalen Ziele, Gegner, Scores, Belohnungen, Level und folgenlosen Wirkungen wurden entfernt. Die verbleibende Architektur steht unabhängig von ihrer Entdeckungsmetapher.

E Wissenschaftliche Einordnung

E.1 Beziehung zu etablierten Linien

OPERATORIUM verbindet, ohne Gleichsetzung zu behaupten:

- ▶ ökologische Interfacegestaltung und constraintbasierte Sichtbarkeit (Vicente und Rasmussen, 1992; Vicente, 2002),
- ▶ Direct Manipulation und kurze Feedbackschleifen (Shneiderman, 1983),
- ▶ Situation Awareness als Wahrnehmung, Verständnis und Projektion (Endsley, 1995),
- ▶ Affordanzen als handlungsbezogene Umweltrelationen (Gibson, 1979; Norman, 2013),
- ▶ verteilte Kognition und externe Repräsentation (Hutchins, 1995),
- ▶ intrinsische Motivation und Kompetenz, ohne daraus Manipulationsziele abzuleiten (Ryan und Deci, 2000),
- ▶ Capability Security für explizite, delegierbare Rechte (Miller, 2006),
- ▶ Safety Engineering und kontrollierte Systemgrenzen (Leveson, 2011),
- ▶ End-to-End-Prüfung unverzichtbarer Tatsachen (Saltzer et al., 1984).

Der eigenständige Beitrag liegt in der gemeinsamen Normalform: Ein Compiler erzeugt aus Blueprint, Principalbindung und Kontext eine versiegelte Operatorgestalt; deren ludomorphe View ist durch semantische Kommutativität und Faktizitätsstatus gebunden; ihre äußere Wirkung wird durch Konsequenzregime, enge Tokens, Receipts und Reconciliation abgeschlossen.

E.2 Abgrenzung zu Digital Twins und Avataren

Ein Digital Twin kann Datenquelle oder Außenpartner sein, ist aber nicht automatisch persongebunden, affordanzgeschlossen oder konsequenzgegate. Ein Avatar kann Erscheinung tragen, besitzt aber nicht notwendig Capability-, Rechte- und Witnesssemantik. OPERATORIUM verlangt diese Verträge und bleibt gegenüber konkreten Twin- oder Avatartechnologien neutral.

E.3 Abgrenzung zu Serious Games

Ein Serious Game ist weiterhin ein Spiel mit einem für Lernen, Training oder Forschung eingesetzten Zweck. OPERATORIUM erzeugt keinen separaten Spielraum: Seine Zustände sind Projektionen realer oder ausdrücklich simulierter Zustände, und seine Wirkungspfade bleiben Teil des tatsächlichen Systems. Die Unterscheidung ist architektonisch, nicht ästhetisch.

Literatur

Literaturverzeichnis

- Deterding, S., Dixon, D., Khaled, R. und Nacke, L. (2011). From game design elements to gamefulness: Defining „gamification“. In *Proceedings of MindTrek 2011*, 9–15. <https://doi.org/10.1145/2181037.2181040>.
- Deterding, S., Sicart, M., Nacke, L., O'Hara, K. und Dixon, D. (2011). Gamification: Using game-design elements in non-gaming contexts. In *CHI 2011 Extended Abstracts*, 2425–2428. <https://doi.org/10.1145/1979742.1979575>.
- Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1), 32–64. <https://doi.org/10.1518/001872095779049543>.
- Gibson, J. J. (1979). *The Ecological Approach to Visual Perception*. Houghton Mifflin, Boston.
- Hutchins, E. (1995). *Cognition in the Wild*. MIT Press, Cambridge, MA.
- Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, Cambridge, MA.
- Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University.
- Norman, D. A. (2013). *The Design of Everyday Things*, revised and expanded edition. Basic Books, New York.
- Ryan, R. M. und Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1), 68–78. <https://doi.org/10.1037/0003-066X.55.1.68>.
- Saltzer, J. H., Reed, D. P. und Clark, D. D. (1984). End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4), 277–288. <https://doi.org/10.1145/357401.357402>.
- Shneiderman, B. (1983). Direct manipulation: A step beyond programming languages. *Computer*, 16(8), 57–69. <https://doi.org/10.1109/MC.1983.1654471>.
- Vicente, K. J. und Rasmussen, J. (1992). Ecological interface design: Theoretical foundations. *IEEE Transactions on Systems, Man, and Cybernetics*, 22(4), 589–606. <https://doi.org/10.1109/21.156574>.
- Vicente, K. J. (2002). Ecological interface design: Progress and challenges. *Human Factors*, 44(1), 62–78. <https://doi.org/10.1518/0018720024494829>.

Sachregister

Affordanz, [33](#)

Altitude, [35](#)

Aufmerksamkeit, [37](#)

Außenvertrag, [7](#)

Blueprint, [21](#)

Capability Envelope, [37](#)

Compiler, [23](#)

Faktizität, [11](#)

Falsifikation, [57](#)

Gamification, [5](#)

Instanz, [25](#)

Invarianten, [55](#)

Konsequenzleiter, [39](#)

Laufnormalform, [47](#)

Laufzustand, [43](#)

Ledger, [49](#)

Ludomorphie, [6](#)

Navigation, [35](#)

Operatorartefakt, [25](#)

Operatoridentität, [15](#)

OPERATORIUM, [3](#)

Operatorleib, [29](#)

Operatormatrix, [17](#)

Operatorregistry, [47](#)

Principal, [15](#)

Projektionsgrammatik, [31](#)

Records, [45](#)

Replay, [49](#)

Ressourcen, [37](#)

semantische Treue, [13](#)

Sicherheit, [51](#)

Souveränität, [51](#)

Trace, [49](#)

Verkörperung, [29](#)

Wirkungsgrenze, [39](#)

OPERATORIUM

The operator is not a character inside a second world.

It is a bounded causal form through which a person can inhabit the first.